OSPREYFLOW

Installation and Administration Guide

OspreyFlow is an open-source intelligence mapping platform for crime analysis, anti-money-laundering, fraud detection, and event monitoring and alerting.

Version	1.27
Date	2026-11-03
Describes	OspreyFlow v0.4.3
Status	Issued

Document control

Version 1.27 · 2026-11-03 · For OspreyFlow v0.4.3 · Issued

Version	Date	What changed
1.27	2026-11-03	"How OspreyFlow is built": a level 0 diagram (who uses OspreyFlow and what it talks to), a level 1 diagram (every component with the technology it is built on, and what each mode runs on), and a new section, "The interfaces, and what crosses each", giving each interface's direction, protocol, what crosses it and what guards it.
1.26	2026-11-02	Checked against the code end to end (build order step 10). §10: Ask to search carries the case's handling and datasets, so the analyst's engine choice applies to it as to Ask the case; it no longer always runs on the local engine.
1.25	2026-11-02	§9.2: the right to manage connection-strength influencers, held by the Analyst and Admin roles. §9.6: approving an installation-wide influencer version, a second person's decision; a case's own influencers need none.
1.24	2026-11-02	§1, "HTTP or HTTPS": Lite now answers on this computer only — every port it publishes is bound to `127.0.0.1`, the Conduit's own port included — so other machines cannot reach it; Small and Enterprise must have an `https://` address, and `osprey.yaml` refuses a plain-HTTP one; a gov.uk address is refused for Lite. §8.1 says the same.
1.23	2026-11-02	§1: a note, "HTTP or HTTPS" — Lite is plain HTTP on a `localhost` address and has no HTTPS option, and its port is published on every network interface, so it is never opened to a network; Small and Enterprise are HTTPS with a certificate Caddy obtains and renews, port 80 only for the certificate and the move to HTTPS; the managed services over TLS; a gov.uk service over HTTPS. §8.1 links to it.
1.22	2026-11-02	New §1a, Choosing a starter: a theme and a domain at install (`--starter`, and the guided page's first screen), the GOV.UK service's name, several deployments side by side (`--name`, `ospreyflow deployments`, `ospreyflow switch <name>`) and how the checkout's own deployment fits in, and a public dataset of your own choosing (`ospreyflow demo add public`) with the VAST Challenge catalogue and its terms. §7: `starter`, `appearance`, `demo_accounts`. §8: `ospreyflow deployments`, `ospreyflow starters`, `--name`. §11: the `cyber-intrusion` pack, and where each starter's demo comes from. Appendix A: `deployment.graph_port`.
1.21	2026-11-01	§9.5: a sixth connector, the FCDO's UK Sanctions List (the XML file, Open Government Licence v3.0); the OFSI connector marked as reading a closed list, on the Lists tab and in the subscribe form. §12: the `closed` section's labels are the five closing reasons as stored.
1.20	2026-11-01	§9.5: the UN and OpenSanctions connectors are marked "licence not confirmed" and read OspreyFlow's synthetic lists only, whatever this device's exceptions admit; the refreshes' counts of customers re-screened and alerts raised are your own unit's.
1.19	2026-11-01	§9.5: a list's refresh schedule, set in Monitoring → Lists and run as the person who set it. §9.6: approval rights — a second person for a rule for the whole deployment and for closing a high-severity alert, and who holds the right to approve; whole-deployment rules live before this version wait for approval after upgrading.
1.18	2026-11-01	§9.5: screening lists — subscribing to a list through its connector, each list's licence, the rule on real lists, and the refresh (*Refresh now*, `python -m scripts.refresh_screening_list`), which re-screens customers against the changed entries only.
1.17	2026-11-01	§12: the monitoring board, and the four monitoring sections the feed carries (`raised_by_rule`, `closed`, `time_to_close`, `hotlist_matches`).
1.16	2026-11-01	§9.4: monitoring, and whom a hotlist's alerts reach (`monitoring.hotlist_audience`, `hotlist_audience_editable`). Appendix A: the `monitoring` settings.
1.15	2026-11-01	§9.1: the Datasets tab. §10: a provider's privacy-contract mark (`privacy_contract`, `contract_reference`); each dataset's egress (`none`, `contracted`, `any`), set in Admin → Datasets and audited; every dataset is `none` after upgrading, so hosted engines receive nothing until an administrator changes one. Appendix A: the two new provider settings.
1.14	2026-11-01	§8: `ospreyflow apply`, and a new §8.3, "Changing a setting". `deployment.env` is readable by its owner alone. §10: a provider enabled on a running deployment, and key entry on the guided install's Models screen.
1.13	2026-11-01	§10 rewritten for several model engines: providers, defaults, labels, keys as `OSPREY_MODEL_KEY_<NAME>`, and what may leave. §8: `ospreyflow models`, `models key` and `models remove-key`. Appendix A: `models.providers`, `models.defaults` and a provider's `label`.
1.12	2026-11-01	The opening says what OspreyFlow is for: intelligence mapping, for crime analysis, anti-money-laundering, fraud detection, and event monitoring and alerting.
1.11	2026-11-01	§8: `ospreyflow switch`. Appendix A: `deployment.name`, the `appearance` settings (the GOV.UK look), and relative storage paths.
1.10	2026-10-29	§11: the Prison Intelligence pack's rehearsal commands are in `DEMO_GUIDE.md`, not the user guide.
1.9	2026-10-29	§11: the two Prison Intelligence packs, the demonstration case and its scale cohort.

Version	Date	What changed
1.8	2026-10-28	Describes v0.4.0. No change to how it is installed, updated or looked after.
1.7	2026-10-27	Appendix A: `components.post_ingest_resolution`, whether a load queues entity resolution afterwards.
1.6	2026-10-18	§11: the HMP pack as a prison intelligence network, and adding it again after an update.
1.5	2026-10-15	A cover page, contents, a list of figures, and this version history.
1.4	2026-10-15	§16: a model download that stopped with `rename …-partial …`, and how to finish it.
1.3	2026-10-15	§11: the Enron email corpus as an optional download, with its safeguards.
1.2	2026-10-14	`ospreyflow repair-sign-in`; no step sends an operator to `make`.
1.1	2026-10-14	§14 rewritten: which installs `update` moves, and checking for and moving to a release.
1.0	2026-10-14	First issue in the install package, with screenshots.
0.4	2026-10-14	Enterprise built from your own computer: `install --mode enterprise --cloud`.
0.3	2026-10-14	Enterprise on Kubernetes: the cluster's namespace, storage classes and managed services.
0.2	2026-10-14	The guided install page ships with each release.
0.1	2026-10-13	Lite and Small, with every setting taken from the deployment's own schema.

Contents

How OspreyFlow is built 6
The interfaces, and what crosses each 8
1. Choosing a mode 9
HTTP or HTTPS 10
1a. Choosing a starter 10
1a.1 Several deployments side by side 11
1a.2 Adding a public dataset 12
2. Before you start 12
2.1 Lite 12
2.2 Small 12
2.3 Enterprise 13
2.4 Checking a machine: ospreyflow doctor 13
3. Getting the ospreyflow command 13
4. Installing Lite 13
4.1 The guided page: ospreyflow install --gui 14
5. Installing Small 14
5.1 What is made in your account 14
5.2 On the server, by hand 15
6. Installing Enterprise 15
6.1 What is made in your account 16
7. The answers file, install.yaml 16
8. Running it 17
8.1 Addresses 17
8.2 Where things are 18
8.3 Changing a setting 18
9. People and accounts 18
9.1 The Admin area 18
9.2 Roles 19
9.3 Inviting someone 19
9.4 Monitoring: whom a hotlist's alerts reach 20
9.5 Screening lists: subscribing, licences and the refresh 20
9.6 Approval rights: a second person 22
10. Models 22
11. Demo packs 23
12. Dashboards 25
The monitoring board 26
13. Backup and restore 27
14. Updates 27
15. Scheduled and watched pipelines 28
16. When something is wrong 29
Appendix A — every osprey.yaml setting 29

List of figures

Figure 1 — Level 0: the people who use OspreyFlow, its three tiers joined by four ports, and the identity provider, reporting, data in and the optional outside services beside it 6
Figure 2 — Level 1: every component by tier with the technology it is built on, the ports between the Conduit and the Vault, identity, reporting, and what each mode runs on 7
Figure 3 — Admin → People and roles: each person's roles, with Access, Disable and Reset sign-in 19
Figure 4 — A person's access opened under their row: clearance, unit and handling codes 20
Figure 5 — Admin → Demo packs: what each pack shows, where it was loaded, and the command for the rest 25
Figure 6 — Admin → Dashboards: registering a dashboard's account and seeing when it last pulled 26
Figure 7 — Superset at bi.: the boards and charts built on the reporting figures 26
Figure 8 — Admin → System: each component's health, the models in use, the version, and the last update check 28
Figure 9 — Admin → Schedules and runs 29

OspreyFlow is an open-source intelligence mapping platform for crime analysis, anti-money-laundering, fraud detection, and event monitoring and alerting.

For whoever installs OspreyFlow and looks after it: **Lite** (one computer), **Small** (one cloud server on the cloud's managed services) and **Enterprise** (a Kubernetes cluster on the cloud's managed services). It describes what is built, and says plainly where something is not yet.

The other guides in this folder are the user guide (user-guide.pdf), for analysts, and the developer guide (developer-guide.pdf), for data engineers. Appendix A, every `osprey.yaml` setting, is made from the deployment's own schema, and is checked against it on every change.

How OspreyFlow is built

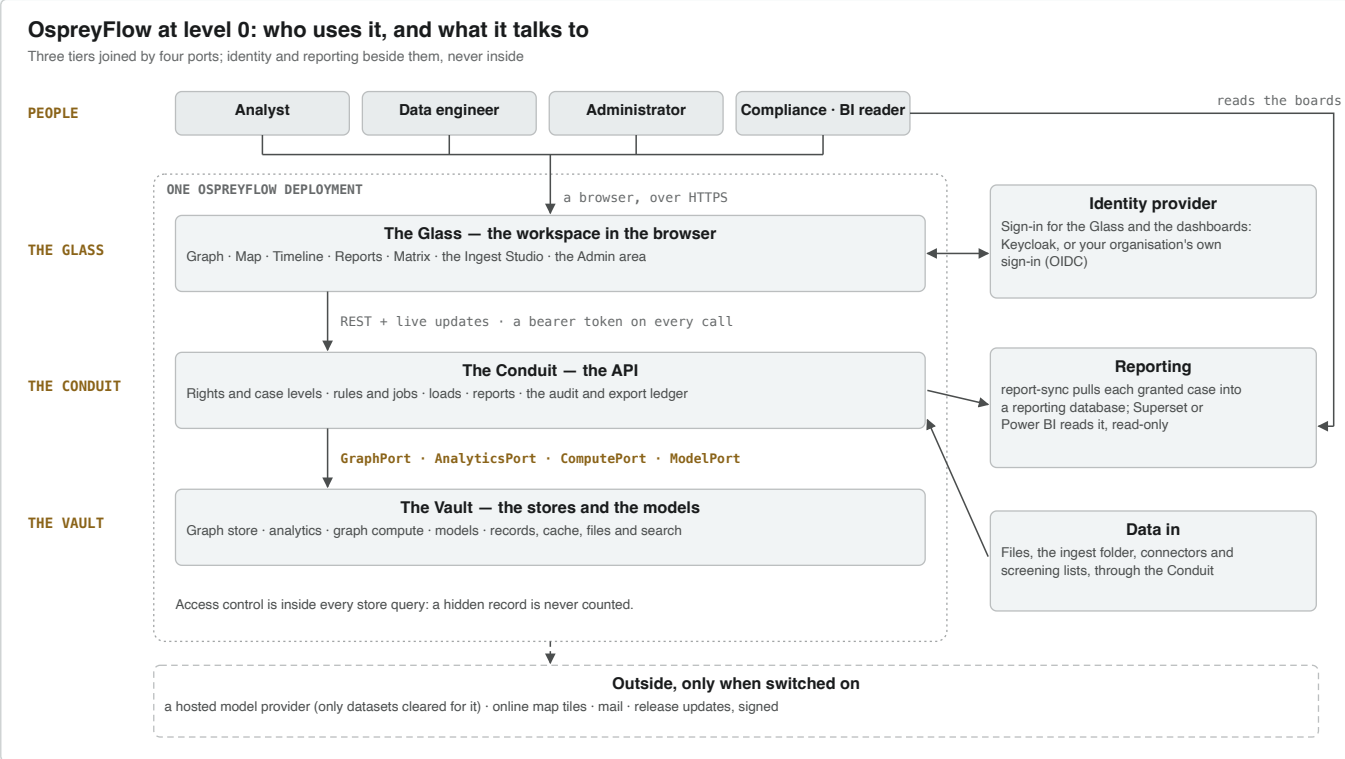


Figure 1. Level 0: the people who use OspreyFlow, its three tiers joined by four ports, and the identity provider, reporting, data in and the optional outside services beside it

Three tiers; everything that passes between them crosses one of four typed ports:

Tier	What it is
The Glass	The browser workspace — the graph, map, timeline and reports, the Ingest Studio and the Admin area — one web application
The Conduit	The API (FastAPI). Every request is checked against the person making it: access control is applied inside each query to the stores, never after
The Vault	The stores: the graph store (ArcadeDB), the analytics database (DuckDB inside the Conduit, or ClickHouse shared by several Conduits), search (OpenSearch, or PostgreSQL full-text), PostgreSQL, the object store, and the models

Port	What passes through it
GraphPort	Typed traversal of the graph, with the person's clearance and handling injected into it
AnalyticsPort	Events, trends and histograms
ComputePort	Graph algorithms: centrality, communities, ownership chains
ModelPort	Extraction, vision, transcription and embedding, through the LiteLLM gateway

Two narrower ports sit beside them: the media store, for content files (an S3-protocol store — SeaweedFS or AWS S3 — or Azure Blob), and the recognition port.

The level 1 diagram names the technology in each component. Versions are those the release pins; Small and Enterprise use the cloud's managed services where the diagram's last row says.

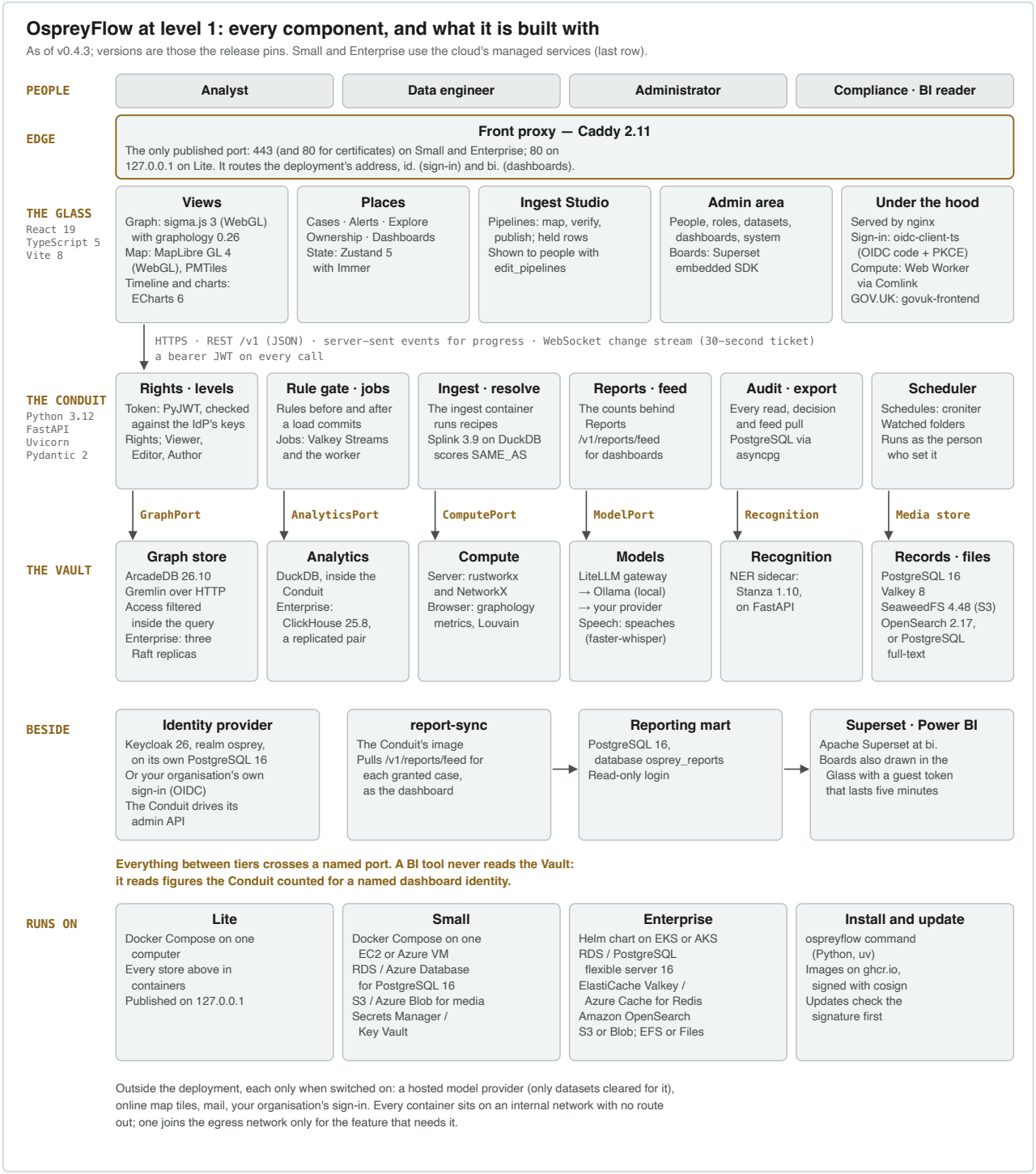


Figure 2. Level 1: every component by tier with the technology it is built on, the ports between the Conduit and the Vault, identity, reporting, and what each mode runs on

One door in. A front proxy (Caddy) is the only published port: 80 on Lite, 80 and 443 with automatic HTTPS on Small and Enterprise. Behind it, at the deployment's address, are the Glass; sign-in (Keycloak) at `id.`; and dashboards (Superset) at `bi.`.

No route out by default. Every container sits on an internal network with no route to the internet. One joins the egress network only when a configured feature needs it: the model and speech-model downloads, the proxy's certificate, a client's own model provider, mail, an organisation's own sign-in, online map tiles, and on Small and Enterprise the cloud's managed services.

What the data is. Model-derived data stays *inferred* until a person confirms it. Records are never merged in storage: matching records are linked with a score and shown as one at query time. Every read is checked against the reader's clearance, unit and handling codes, and a hidden record cannot be detected from a count, a path length or a chart.

The interfaces, and what crosses each

Each interface has one direction, one owner and one guard. None of them lets a store's own query language, a source system's identifier or an unfiltered count reach the Glass.

Interface	From → to	How	What crosses	Guarded by
The front door	Browser → front proxy (Caddy) → the Glass, the Conduit (`/v1`, through the Glass's nginx), sign-in at `id.`, dashboards at `bi.`	HTTPS on 443, with 80 only for the certificate and the move to HTTPS (Small, Enterprise); HTTP on 80, bound to `127.0.0.1` (Lite)	Every request	The only published port; every other container is on an internal network
Sign-in and token	The Glass, Superset → identity provider → the Conduit	OpenID Connect, authorisation code with PKCE; a signed JWT access token	Credentials, typed only at the identity provider; then who the person is, their clearance, unit, handling codes and roles	Every token's signature checked against the provider's keys on every call
Rights	The Conduit → the Glass (`GET /v1/me/rights`)	REST	What this person may do; the rail and menus follow it	Each route checks its own right, so the Conduit refuses whatever a stale screen offers
The API	The Glass → the Conduit (`/v1/…`)	HTTPS, JSON, a bearer token on every call	Searches, working sets, edits, loads, reports, exports	Access control inside each store query; a result too large is refused with `limit_exceeded`, never cut silently
Live updates	The Conduit → the Glass	WebSocket `/v1/investigations/{id}/stream`, opened with a one-time ticket that lasts 30 seconds; server-sent events for long-running progress	Changes to the open case; progress	The ticket is the person's own; the same access as the API
Administration	The Admin area → the Conduit (`/v1/admin/…`) → the identity provider's admin API	REST	People, roles and access; datasets and their egress; dashboard identities; schedules; health and freshness	Administration rights (`manage_people`, `manage_dashboards`, `administer_system` and others); a dashboard is never given more than its registrar holds
Case access	The Share panel → the Conduit (`/v1/investigations/{id}/access`)	REST	Who holds Viewer, Editor or Author on the case, by account id	The case's Author; every change kept in the case's history
GraphPort	The Conduit → the graph store (ArcadeDB)	Gremlin over ArcadeDB's HTTP API, on the internal network only	Typed traversals; writes; a load's revert	The person's clearance and handling codes and the case's datasets are part of the traversal itself
AnalyticsPort	The Conduit → DuckDB (inside the Conduit) or ClickHouse (HTTP)	In process, or HTTP on the internal network	Event aggregates, trends and histograms; never raw SQL from the Glass	The same filters inside the query; no count includes a record the reader cannot see
ComputePort	The Conduit → rustworkx and NetworkX; the Glass → graphology in a Web Worker	In process	Centrality, communities, ownership chains	Computed over what the reader can see only; the two engines are held to the same conformance suite

Interface	From → to	How	What crosses	Guarded by
ModelPort	The Conduit → LiteLLM → Ollama, speaches, or a provider you configure	HTTP, OpenAI-compatible	Text, images and audio to extract from, describe, transcribe or embed; what returns is *inferred* until a person confirms it	Each dataset's egress (`none`, `contracted`, `any`) and the handling codes that may be sent (§10)
Recognition	The Conduit → the NER sidecar (Stanza)	HTTP on the internal network	Text to tag with names, places and organisations	No route out; what it finds is *inferred*
Media store	The Conduit and the ingest container → SeaweedFS, S3 or Azure Blob	The S3 protocol, or Blob's API	Content files, staging, site images, the tile cache	Private, encrypted storage; files reach the Glass only through the Conduit
Records, cache and search	The Conduit → PostgreSQL, Valkey, OpenSearch	The internal network (TLS and a token on Enterprise's managed cache)	The Conduit's own records and audit; caches, the job queue and the change stream; text search	Reached only from the deployment's own network
Ingest	The Ingest Studio, the ingest folder and connectors → the Conduit → the ingest container	REST; a watched folder	Files and rows, loaded under a published recipe	`edit_pipelines` and `schedule_pipelines`; a run is made as the person who set it; the case's rules judge each load before and after it commits, and hold the rows they refuse
Reporting feed	report-sync → the Conduit (`/v1/reports/feed`)	REST, with the dashboard identity's own client credentials	Figures only, counted as that dashboard	The dashboard's own token and the cases it was given; every pull in the export ledger
Reporting mart	report-sync → PostgreSQL `osprey_reports` → Superset or another BI tool	SQL, a read-only login	The feed's rows; pull times and failures	A read-only login; never the Conduit's own tables
Embedded board	The Glass → the Conduit → Superset	Superset's embedded SDK, with a read-only guest token that lasts five minutes	One board	Given only to a viewer who could count every figure on it
Updates	`ospreyflow update` → the repository's releases on GitHub and its images on `ghcr.io`	HTTPS	The list of releases, the release's lock and its images	The release's signature, made by this repository's own release workflow, checked with cosign before anything changes (§14)
Outside services	A container on the egress network → model and speech-model downloads, a provider you configure, mail, your organisation's sign-in, online map tiles, the cloud's managed services	HTTPS	Only what the feature needs	A container joins the egress network only when its feature is switched on

1. Choosing a mode

	Lite	Small	Enterprise
Where it runs	One computer (a laptop or a workstation)	One cloud server (AWS or Azure)	A Kubernetes cluster (EKS or AKS), three or more nodes

	Lite	Small	Enterprise
Who looks after the data	You, with `ospreyflow backup`	The cloud: managed PostgreSQL, S3 or Blob, daily backups	The cloud, for its managed services; the graph store in the cluster is not yet backed up (section 13)
Its address	`http://ospreyflow.localhost`	`https://<your domain>`	`https://<your domain>`
First sign-in	Demo accounts, passwords in `deployment.env`	A first administrator, by a one-time code	A first administrator, by a one-time code
Built with	`ospreyflow install --mode lite`	`ospreyflow install --mode small --cloud aws|azure`, from your own computer	`ospreyflow install --mode enterprise --cloud aws|azure`, from your own computer, or onto a cluster you already have

All three run the same product: the same containers, the same Glass, Ingest Studio and Admin area. Small swaps the deployment's own PostgreSQL and object store for the cloud's, and gets a real domain with HTTPS. Enterprise runs the same services as replicas across the cluster's nodes, the graph store and the analytics database as clusters of their own.

HTTP or HTTPS

- **Lite serves plain HTTP** at a `localhost` address: `http://ospreyflow.localhost`, or a named deployment's `http://<name>.localhost:<port>` (section 1a.1). That is deliberate. Lite is for one computer, its traffic is meant to stay on that machine, and browsers treat `localhost` as a secure context, so sign-in works without a certificate.
- **Lite has no HTTPS option.** The install, the guided page and every starter give it an `http://…localhost` address, and Lite is not built or tested with a certificate.
- **Lite answers on this computer only.** Every port a Lite deployment publishes — the front proxy's, the graph store's and, when `network.expose_conduit` is on, the Conduit's — is bound to this computer's own address (`127.0.0.1`), so other machines on a network cannot reach it, whatever the firewall allows. A deployment kept by name (section 1a.1) is Lite, so the same holds. Use Small or Enterprise for anything other people reach.
- **Small and Enterprise serve HTTPS** at `https://<your domain>`, and must: `osprey.yaml` refuses a Small or Enterprise deployment whose `network.public_origin` is not an `https://` address, and `ospreyflow apply`, the guided page and the Set-up console say so and change nothing. The front proxy (Caddy) obtains the certificate for your domain and renews it automatically. Port 80 is open only for the certificate's challenge and to send a request for your domain on to HTTPS (443); any other request on port 80 gets "not found", apart from the proxy's own health check. On Enterprise, the cluster's load balancer passes 80 and 443 to the proxy.
- **The managed services behind Small and Enterprise are reached over TLS:** PostgreSQL always (`sslmode` is `require` at least), the object store always over HTTPS, and the cache and the mail server over TLS unless you turn it off (`external.*` and `mail.starttls`, Appendix A).
- **A service on a gov.uk domain must be served over HTTPS,** the GOV.UK theme included, so it is Small or Enterprise, never Lite: `osprey.yaml` refuses a Lite deployment at a gov.uk address. The GOV.UK theme on a `localhost` address stays available for a demonstration on one computer.

1a. Choosing a starter

A **starter** is a theme and a domain, chosen at install. It only fills in answers the install already asks — the look, the GOV.UK service's name, organisation and phase, the demo packs and whether demo accounts are made — and every one can still be changed. The `osprey.yaml` it makes records the starter in a comment at the top; nothing at run time reads it. Every domain pack (FIBO, NIEM, STIX) stays available to every case whatever the starter: the domain chooses the demo data and what its demo cases are set to, never what the ontology offers.

	`crime`	`aml`	`fraud`	`cyber`	`blank`
Demo case's domains	NIEM	FIBO	FIBO	STIX	none chosen
Demo data	`prison-intelligence`, `hmp`	`operation-ledger`, with its synthetic screening lists	`fixture-corp`; PaySim offered as your own download, never fetched	`cyber-intrusion`	none, and no demo accounts
`ospreyflow` theme (the base look)	`ospreyflow-crime`	`ospreyflow-aml`	`ospreyflow-fraud`	`ospreyflow-cyber`	`ospreyflow-blank`
`govuk` theme (GOV.UK, a gov.uk service only)	`govuk-crime`	`govuk-aml`	`govuk-fraud`	`govuk-cyber`	`govuk-blank`

`ospreyflow starters` (or `ospreyflow install --list-starters`) prints the same table and the command for each.

One command per starter. Add `--yes` to take every other default; leave it out to be asked.

```
ospreyflow install --mode lite --starter ospreyflow-crime
ospreyflow install --mode lite --starter ospreyflow-aml
ospreyflow install --mode lite --starter ospreyflow-fraud
ospreyflow install --mode lite --starter ospreyflow-cyber
ospreyflow install --mode lite --starter ospreyflow-blank
ospreyflow install --mode lite --starter govuk-crime --service-name "Analyse crime intelligence" --
organisation "<your organisation>"
ospreyflow install --mode lite --starter govuk-aml --service-name "Investigate money laundering" --
organisation "<your organisation>"
ospreyflow install --mode lite --starter govuk-fraud --service-name "Investigate fraud" --organisation "
<your organisation>"
ospreyflow install --mode lite --starter govuk-cyber --service-name "Investigate cyber incidents" --
organisation "<your organisation>"
ospreyflow install --mode lite --starter govuk-blank --service-name "<your service's name>" --organisation
"<your organisation>"
```

The GOV.UK theme asks for the service's name, organisation and phase (`--service-name`, `--organisation`, `--phase alpha|beta|live|none`; the phase offered first is `beta`). Without a service name the install is refused, in words, before anything is changed. **Name the service, not a case:** the GOV.UK header shows the name on every page of every case, so the names offered above describe the whole deployment; "Prison Intelligence" in the header of the Ledger's case is the mistake this avoids. `blank` offers no name: a real service names itself.

On the guided page (`ospreyflow install --gui`, section 4.1) the starter is on the first screen, Mode, under **Starter (optional)**; a GOV.UK starter shows the service's name (filled in with the name above), organisation and phase there. The Demo packs screen then shows the starter's packs ticked, to change if you wish.

In an answers file (section 7): `starter: govuk-aml`, with `appearance.service_name`, `appearance.organisation` and `appearance.phase`.

1a.1 Several deployments side by side

`--name` keeps a deployment of its own beside any other on the same computer:

```
ospreyflow install --mode lite --name crime-demo --starter ospreyflow-crime
ospreyflow install --mode lite --name aml-govuk --starter govuk-aml --service-name "Investigate money
laundering" --organisation "<your organisation>"
```

Each one has:

- its own folder, `~/.ospreyflow/deployments/<name>/`, holding its `osprey.yaml`, every file rendered from it and its `deployment.env` (readable by your account alone, `600`; the folder is `700`);
- its own Compose project and volumes, named `<name>`: no case, account or index is shared;
- its own address — `--address`, by default `http://<name>.localhost:<the next free port>` from 8090 — and its own graph-store port on this computer (`deployment.graph_port`, from 2500).

A name that Docker already knows as another folder's Compose project is refused, before anything is written: two deployments under one name would share containers. The checkout itself is shared; no second checkout or worktree is needed.

Every command takes `--name`: `ospreyflow start --name crime-demo`, `ospreyflow status --name aml-govuk`, `ospreyflow demo add hmp --name crime-demo`, `ospreyflow backup --name aml-govuk` (whose backups are kept in the deployment's own folder). Without `--name`, a command acts on the deployment last used, or on the only one there is.

```
ospreyflow deployments          # each one's name, starter, address and whether it is running
ospreyflow switch aml-govuk     # stop the running one, start aml-govuk, open it
ospreyflow switch default       # back to the checkout's own
```

A Lite computer runs one deployment at a time by default, which is what `ospreyflow switch` is for.

The deployment installed in the checkout — every install before this version, with its `osprey.yaml` in the checkout and its files in `deploy/generated/` — is listed as `default` and works exactly as before: no command moves, renames or changes it. It could

later be moved under `~/.ospreyflow/deployments/` (install a named deployment, restore a backup of the checkout's into it, check it, then uninstall the checkout's with `--keep-data` until you are sure); that is a step for the administrator to take deliberately, never something an update does.

1a.2 Adding a public dataset

A public dataset you download yourself can be added to any deployment, at install or later:

```
ospreyflow demo add public --file <the downloaded file> --manifest <its dataset.yaml>
```

A **manifest** says what the dataset is, its source page, its licence or terms as the source states them, the file's SHA-256, whether it holds real people's data, and the Ingest Studio recipe that loads it. The file is checked against the checksum before anything else happens; a file that differs is refused, and nothing is loaded. It loads into a case of its own, named after the dataset.

Real personal data (`real_personal_data: true` in its manifest) is loaded only once you type `yes`, accepting that it is real people's data and that your organisation has its own lawful basis to process it — as for Enron (section 11); `--accept-real-data` accepts in a scripted run. It never belongs in a repository or a test run.

The VAST Challenge catalogue. OspreyFlow ships manifests and recipes — never the data — for two IEEE VAST Challenge datasets, fictional scenarios published for visual-analytics research:

Catalogue id	Dataset	Suits	Source
`vast-2012-mc2`	VAST Challenge 2012, Mini-Challenge 2: the fictional Bank of Money's regional network — its firewall log (`Firewall-04072012.csv`)	the `cyber` starter (its case is set to STIX)	the Visual Analytics Benchmark Repository, University of Maryland
`vast-2008-mc3`	VAST Challenge 2008, Mini Challenge 3: cell phone call records on the fictional island of Isla del Sueño (synthetic)	the `crime` starter (its case is set to NIEM)	the same repository

Their terms. Neither the source pages, nor the challenges' task pages, nor the repository's mirror at Washington University state any licence or terms of use (checked 2 November 2026). No licence means no right to redistribute is granted: download the data yourself, use it on your own computer, never pass it on, and do not rely on it for commercial use without the publishers' permission. Each manifest says so.

Their checksums. OspreyFlow never downloads VAST data, so the catalogue records no checksum, and `demo add public` refuses an entry until you record the checksum of the file you downloaded:

```
python -m scripts.public_dataset --list
python -m scripts.public_dataset pin --manifest vast-2012-mc2 --file Firewall-04072012.csv --out
~/OspreyFlow/vast-2012-mc2.yaml
ospreyflow demo add public --file Firewall-04072012.csv --manifest ~/OspreyFlow/vast-2012-mc2.yaml --name
crime-demo
```

`pin` writes your own copy of the manifest, with the recipe beside it; check first that the file came from the source page, since what you pin is what every later check holds the file to. The column layout each recipe reads is the challenge's published one; check it against your download's first line.

2. Before you start

2.1 Lite

- **Docker**: Docker Desktop on macOS and Windows (on Windows with the WSL 2 back end); Docker Engine with the Compose v2 plugin on Linux. Set it to start when you sign in: the stack comes back after a reboot only once Docker does.
- **Memory**: at least 16 GB available to Docker (32 GB is comfortable).
- **Disk**: at least 60 GB free, plus the demo packs you choose (PaySim is about 0.5 GB to download and more once loaded).
- **Git**, and **internet** for the first install.
- **Port 80 free** (the deployment's one door).

2.2 Small

On your own computer, from which you build the server:

- an **AWS or Azure account** you may create resources in;
- **OpenTofu** (`tofu`), which builds the server;
- the cloud's **command line** (`aws` or `az`), signed in to that account;
- **Git** and **uv**;
- a **DNS name** you control, such as `osprey.example.org`. If the name's zone is in the same cloud account (Route 53 or Azure DNS), the records are made for you.

The server itself needs nothing from you: it installs Docker and OspreyFlow on its first boot.

2.3 Enterprise

On your own computer:

- what Small needs (an account, OpenTofu, the cloud's command line signed in, Git, uv, a DNS name);
- **kubectl** and **Helm**;
- on Azure, **kubelogin** (`az aks install-cli` installs it): the cluster takes Entra ID sign-ins only;
- the **addresses you work from** (`api_cidrs`): the cluster's API, and on Azure the Key Vault, answer those and nothing else, so this computer must be inside one of them.

Onto a cluster you already have, only kubectl, Helm, Git and uv are needed, with kubectl pointed at it.

2.4 Checking a machine: `ospreyflow doctor`

`ospreyflow doctor --mode lite` checks, in the order to fix them: Docker installed, Docker running, Compose v2, memory, free disk, Git, uv and port 80. `ospreyflow doctor --mode small --cloud aws` (or `azure`) checks Git, uv, OpenTofu and the cloud's command line. `ospreyflow doctor --mode enterprise --cloud aws` adds kubectl and Helm (and on Azure kubelogin); without `--cloud` it checks kubectl, Helm and that a cluster answers. Each line is a tick, or a cross with what to do about it; the exit code is 1 when anything is missing. `install` runs the same checks first and stops, having changed nothing, if one fails.

3. Getting the `ospreyflow` command

The command is installed once per computer by the bootstrap in the repository:

- **macOS and Linux**: `sh install.sh` from inside a checkout, or `OSPREYFLOW_REPO=<repository address> sh install.sh` to clone it into `~/ospreyflow`.
- **Windows**: `install.ps1` in PowerShell, the same way (`$HOME\ospreyflow`).

It installs **uv** if it is missing (from Astral's own installer), makes the checkout's Python environment, installs the command as a uv tool, and records where the checkout is (`~/.config/ospreyflow/home`, or `%APPDATA%\ospreyflow\home`). It starts nothing. If the command is not found afterwards, open a new terminal, or run `uv tool update-shell`.

The command finds its checkout by `OSPREYFLOW_HOME` first, then the recorded file, then by looking upwards from the folder you are in. Anything it prints that names a password, secret or token is hidden and replaced by a pointer to `deployment.env`.

4. Installing Lite

```
ospreyflow install --mode lite
```

asks a few questions — dashboards on or off, the folder the Ingest Studio lists, the folder for a basemap, which demo packs — then (`--starter <theme>-<domain>` fills these in first: section 1a):

1. writes `osprey.yaml` from the answers (refusing, with nothing changed, if a different `osprey.yaml` is already there, unless `--force`);
2. on a release (the checkout sits on a `vX.Y.Z` tag): checks the release's signatures and runs exactly its images, pinned by digest; otherwise builds the images from the checkout;
3. starts the deployment and waits until every service is healthy;
4. makes the demo accounts (unless `--no-demo-accounts`) and loads the chosen demo packs.

It ends by saying the address and that the sign-in details are in `deployment.env`.

Without questions: `ospreyflow install --mode lite --yes` takes every default; `--config install.yaml` takes the answers from a file (section 7). **Building anyway**: `--build` builds from the checkout even on a release tag.

4.1 The guided page: `ospreyflow install --gui`

Opens a page in your browser, on this computer only, behind a one-time link (anyone else, or any other link, is refused). It walks through the same answers as screens, with a starter (section 1a) offered on the first, Mode:

- **Lite**: Mode, Folders, Profile, Components, Map (register a local basemap, build it, verify it), Models, Governance, Demo packs, Review (which runs the doctor), then Install, with the install's own output shown live.
- **Small and Enterprise**: Mode, Cloud and region, Domain, Components, Models, Governance, Demo packs, Review — which **saves** the answers: a Small server is built from a terminal (section 5), never from the page.

Everything is worked on a draft copy of `osprey.yaml`; the page closes itself when the install finishes, or after 30 idle minutes. On a release, the page is the release's own — fetched the first time, with its signature checked against the release; in a developer's checkout it is built with `cd apps/harness && npm ci && npm run build`, and the command says so if it is missing or out of date.

5. Installing Small

From your own computer:

```
ospreyflow install --mode small --cloud aws --config install.yaml
```

with at least:

```
mode: small
cloud:
  provider: aws            # or azure
  region: eu-west-2        # or uksouth, …
  domain: osprey.example.org
  dns_zone: example.org    # optional: make the DNS records in this Route 53 / Azure DNS zone
  admin_email: ops@example.org   # optional
  size: small              # small (32 GB server), medium (64 GB) or large (128 GB)
```

What happens:

1. **Plan.** OpenTofu shows everything it would make in your account (below). Nothing is made yet.
2. **Build, on your yes.** Answer `yes` (or pass `--yes`) and OpenTofu builds it. Its state is kept in `.cloud/small-<cloud>/` in your checkout, readable only by you. **Keep it, privately**: it is how the server is changed later, and on Azure it holds the database password.
3. **DNS.** With a `dns_zone`, the three records (`osprey.example.org`, `id.…` and `bi.…`) are made for you. Without one, the command prints the address to point them at and waits — up to an hour — until the name resolves to the server.
4. **The server installs itself.** On first boot it installs Docker and OspreyFlow, reads the database password (and on Azure the storage key) with **its own cloud identity**, installs onto the managed services, and gets its HTTPS certificate. The command waits — up to 45 minutes — until `https://<domain>` answers. The server's own log is `/var/log/ospreyflow-first-boot.log`.
5. **The first administrator.** The server makes an account `admin` holding the Admin role and the highest clearance, with a **one-time code** instead of a password, and leaves the code in the cloud's secret store. The command reads it, **shows it once**, and deletes it from the store. At their first sign-in the administrator chooses their own password. Then they invite everyone else from the Admin area (section 9).

5.1 What is made in your account

	AWS	Azure
Server	One EC2 instance (r6i), encrypted disk, no SSH; reach it with Session Manager	One VM (E-series), no SSH port; Azure's service healing
Database	RDS for PostgreSQL 16, private, encrypted, its password kept by Secrets Manager	Azure Database for PostgreSQL, private, its password in Key Vault
Objects	S3: media, staging, site images, tile cache (private, encrypted)	Blob for media; the other stores on the server (SeaweedFS), covered by the server's backup
Secrets	Secrets Manager	Key Vault (purge protection on)

	AWS	Azure
Recovery	EC2 automatic recovery; AWS Backup daily	Azure Backup daily
Open to the internet	80 and 443 only	80 and 443 only

Backups are kept 14 days by default (`backup_days`, 7–35).

5.2 On the server, by hand

The server's first boot runs, in effect, `ospreyflow install --mode small --config /etc/ospreyflow/install.yaml --yes --force --first-admin-code-file <file>` with the passwords in its environment. Run without `--first-admin-code-file`, it prints the first administrator's code once instead. `install --mode small` without `--cloud` and without the managed services in its answers is refused, saying which of the two you meant.

6. Installing Enterprise

From your own computer:

```
ospreyflow install --mode enterprise --cloud aws --config install.yaml
```

with at least:

```
mode: enterprise
cloud:
  provider: aws            # or azure
  region: eu-west-2        # or uksouth, …
  domain: osprey.example.org
  api_cidrs: [203.0.113.0/24]    # the addresses you work from; never 0.0.0.0/0
  admin_email: ops@example.org   # optional
  size: small              # small (3 nodes of 32 GB), medium (4 of 64 GB) or large (6 of 128 GB)
cluster:
  namespace: ospreyflow    # optional
```

What happens:

1. **Plan, then build on your yes**, as for Small (section 5): OpenTofu shows the cluster and the services it would make, and makes them when you answer `yes` (or pass `--yes`). Its state is kept in `.cloud/enterprise-<cloud>/`, readable only by you. **Keep it, privately**: it is how the cluster is changed later, and it holds secrets.
2. **What OpenTofu made becomes the answers.** It names the cluster's two storage classes and every managed service; these join your answers in `osprey.yaml`. Do not give `external:` or the storage classes yourself: with `--cloud` they are refused, since OpenTofu makes them.
3. **A kubeconfig of the install's own**, in the state folder; your `~/.kube/config` is not changed. Use it yourself with `KUBECONFIG=.cloud/enterprise-<cloud>/kubeconfig kubectl get pods -n ospreyflow`.
4. **The passwords and keys** of the managed services are read from the cloud's secret store with your own identity and handed to the install in its environment only — never on a command line, in `osprey.yaml` or on the screen.
5. **The install** puts the chart onto the cluster, makes its Secret (keeping every value already there when run again), configures sign-in, makes the first administrator, prints their one-time code once, and says the address to point `osprey.example.org`, `id.…` and `bi.…` at.

Run again, it changes nothing OpenTofu need not change, no password and no account.

Onto a cluster you already have: leave out `--cloud`, point kubectl at the cluster, and give the managed services yourself under `external:` and the cluster's classes under `cluster:` (section 7), with their passwords in the environment: `OSPREY_POSTGRES_PASSWORD`, `VALKEY_PASSWORD`, and for media on Blob `AZURE_STORAGE_KEY`. A site's own Helm values go in with `--chart-values <file>` (repeatable), applied over the rendered ones.

6.1 What is made in your account

	AWS	Azure
Cluster	EKS, its Kubernetes secrets encrypted with their own key; nodes in three private subnets	AKS, no local accounts (Entra ID only); a system pool and the deployment's own pool, three zones
Its API	Answers `api_cidrs` only	Answers `api_cidrs` only
Network policy	The VPC CNI's engine, so the chart's policies are enforced	Azure network policy
Database	RDS for PostgreSQL 16, two zones, password in Secrets Manager, deletion protection	PostgreSQL flexible server 16, zone-redundant, a delete lock, password in Key Vault
Cache	ElastiCache for Valkey, TLS and a token	Azure Cache for Redis, TLS only
Objects	S3 (as Small)	Blob for media; the other stores in the cluster
Search	Amazon OpenSearch Service (the install always asks for it; the module's own `search = false` leaves it to the cluster)	In the cluster
Shared volume	EFS (`ospreyflow-shared`)	Azure Files (`ospreyflow-shared`)
Store volumes	Encrypted gp3 (`ospreyflow-volumes`)	Zone-redundant premium disks (`ospreyflow-volumes`)

Every managed store is reached only from inside the cluster's network. Neither module has yet been built in a real account; the chart and the install have been proved in a local cluster.

7. The answers file, `install.yaml`

Every question `install` asks can be answered in this file; unknown settings are refused by name.

Setting	Meaning
`mode`	`lite`, `small` or `enterprise`
`components.dashboards`	Superset and report sync on (`true`, the default) or off
`folders.ingest`	The folder the Ingest Studio lists (absolute, or from `~`); default `~/OspreyFlow/ingest`
`folders.tiles`	Where a built basemap lives; default `~/OspreyFlow/basemap`
`models.local_only`	`true` (default): no case content leaves the deployment
`demo_packs`	A list of pack ids (section 11)
`paysim_file`	The downloaded PaySim file, when `paysim` is chosen
`mail.host`, `mail.port`, `mail.from`, `mail.starttls`, `mail.username`	A mail server for invitations (section 9.3); its password is never here
`cloud.provider`, `cloud.region`, `cloud.domain`, `cloud.size`, `cloud.dns_zone`, `cloud.admin_email`	Small (section 5) and Enterprise (section 6)
`cloud.api_cidrs`	Enterprise with `--cloud`: the addresses the cluster's API answers (section 6)
`cluster.namespace`, `cluster.storage_class`, `cluster.shared_storage_class`	Enterprise: the namespace, and onto a cluster you already have its storage classes (section 6)
`external`	The managed services the deployment runs on — written by OpenTofu, or by you for a cluster you already have
`config`	Use this `osprey.yaml` exactly as it is, instead of `components`, `folders` and `models`
`starter`	A starter, `<theme>-<domain>` (section 1a): fills in the answers below that are not given
`appearance.brand`, `appearance.service_name`, `appearance.organisation`, `appearance.phase`	The look (appendix A's `appearance`); `govuk` needs `service_name`, which names the whole service
`demo_accounts`	`true` (default) makes the demo accounts; `false` makes none (the `blank` starter's answer)

The answers become `osprey.yaml` (appendix A): `mode` sets `profile`, `components.dashboards` sets both `reporting.sync` and `reporting.superset`, and the address is `http://ospreyflow.localhost` for Lite and `https://<domain>` for Small and Enterprise.

8. Running it

Command	What it does
`ospreyflow start`	Brings the deployment up and waits for every service; `--build` rebuilds from the checkout first
`ospreyflow stop`	Stops every container; nothing is removed
`ospreyflow switch [<name>]`	Makes this the deployment running: stops any other OspreyFlow deployment on the machine (each with its own settings), then starts and opens the named one (`default` for the checkout's; section 1a.1). `--list` names each deployment, running or stopped, and changes nothing. A deployment in another checkout is still picked with `OSPREYFLOW_HOME=<its product folder> ospreyflow switch`
`ospreyflow deployments`	Lists this computer's deployments — the checkout's as `default`, and each kept by name — with the starter each was made from, its address and whether it is running
`ospreyflow starters`	Lists the starters, and the install command for each (section 1a)
`ospreyflow status`	Each service's state and health; exits 1 unless everything is running and healthy
`ospreyflow open`	Opens the Glass in your browser
`ospreyflow logs [service] [--follow] [--tail N]`	The services' logs (the last 200 lines by default)
`ospreyflow apply`	Applies a changed `osprey.yaml` to the installed deployment (section 8.3): renders it again, keeps every password and key, asks for a newly enabled provider's key (from `OSPREY_MODEL_KEY_<NAME>` or a hidden prompt), and restarts only the services whose settings changed. It makes no account, adds no demo pack and touches no data
`ospreyflow repair-sign-in [--no-demo-accounts]`	Registers the sign-in again — the identity provider's clients, roles and the demo accounts (leave them out with `--no-demo-accounts`); safe to run again
`ospreyflow models`	Lists the model engines, each role's default engine, and whether each provider's key is present (yes or no — never the key)
`ospreyflow models key <name>`	Sets or replaces a provider's key, from `OSPREY_MODEL_KEY_<NAME>` in the terminal or a hidden prompt; writes it to `deployment.env` alone and restarts the model gateway (and the Conduit when the key's presence changes)
`ospreyflow models remove-key <name>`	Removes a provider's key; the provider stays enabled but shows "no key" and is not offered to analysts
`ospreyflow uninstall [--keep-data] [--yes]`	Removes the containers; with `--keep-data` the data volumes stay, without it cases, accounts and indexes go too. Your checkout, `osprey.yaml` and your own folders are never touched

Every command takes `--name <name>` to act on a deployment kept by name, or `--name default` for the checkout's own (section 1a.1). Every service restarts itself if it stops; nothing is registered with the operating system.

Enterprise is looked after with Kubernetes' own tools, not these verbs (which drive a Compose deployment): `kubectl get pods -n ospreyflow`, `kubectl logs`, and `helm uninstall ospreyflow -n ospreyflow` to remove it (the stores' volume claims are left in place, and the classes OpenTofu makes keep a volume even when its claim is deleted).

8.1 Addresses

One front proxy is the deployment's only door. Under the address in `network.public_origin`:

- the **Glass** — with the Ingest Studio and the Admin area inside it — at the name itself;
- **sign-in** (Keycloak) at `id.`;
- **dashboards** (Superset) at `bi.`, when dashboards are on.

Lite publishes port 80 (or its own address's port) and `2480` for the graph store's own tools, both on this computer only (`127.0.0.1`): other machines cannot reach a Lite deployment. Small publishes 80 (for the certificate and the move to HTTPS) and 443 on every network interface, because other people reach it. Which of them is plain HTTP and which HTTPS: the note **HTTP or HTTPS** in section 1. The Conduit's own port is published only with `network.expose_conduit: true` — on Lite, on this computer only.

8.2 Where things are

Where	What
`osprey.yaml` (the checkout)	The deployment's settings; never a secret
`~/.ospreyflow/deployments/<name>/`	A deployment kept by name (section 1a.1): its own `osprey.yaml`, every file rendered from it, its `deployment.env` (`600`) and its `backups/`; `~/.ospreyflow/current` names the one last used
`deploy/generated/<profile>/`	The deployment: `compose.yaml`, `deployment.env` (every password and key it made), `Caddyfile`, `glass.json`, `realm.json`, `litellm.yaml`, `versions.lock`, `RUNBOOK.md`, `state/`
`backups/`	Backup sets
`.install/`	The guided page's draft and answers; a release install's verified lock
`.cloud/`	OpenTofu's state for a Small server or an Enterprise cluster, and the cluster's kubeconfig — keep it private
Your ingest and basemap folders	As the answers named them

`RUNBOOK.md`, written for each deployment, lists what runs, its paths and its addresses.

`deployment.env` is readable by your own account alone (`600`): `ospreyflow install` writes it so, and `ospreyflow apply`, `models key` and `models remove-key` keep it so.

8.3 Changing a setting

Edit `osprey.yaml` in the checkout, then run:

```
ospreyflow apply
```

It checks the file first, and if anything in it is wrong it says what, and changes nothing. Then it renders the deployment again from it, keeping every password and key the deployment already has, and starts again only the services whose settings changed; the rest keep running. A provider newly enabled under `models.providers` needs its key: give it as `OSPREY_MODEL_KEY_<NAME>` in the same terminal, or type it at the hidden prompt `apply` shows. `apply` makes no account, adds no demo pack, rebuilds nothing and never touches the stored data. Do not run `ospreyflow install` again over a running deployment to change a setting; `ospreyflow start --build` rebuilds the code but reads the settings the deployment already has.

A changed `deployment.name` names a different deployment, which `apply` refuses: install it as a new one.

9. People and accounts

Everyone signs in through the deployment's identity provider; the password is only ever typed on its own sign-in page. What a person may do is their **roles**; what they may see is their **clearance**, unit and handling codes, and the cases they are given.

9.1 The Admin area

Signed in with an administration right, choose **Admin** on Start. Each tab shows only to someone holding its right:

Tab	Right	What it does
People and roles	`manage_people`	Invite people; set their roles, clearance, unit and handling codes; disable, re-enable; reset a sign-in
Dashboards	`manage_dashboards`	Register a dashboard's account (its secret shown once), see when it last pulled, remove it
Schedules and runs	`operate_schedules`	Every scheduled or watched pipeline: pause, resume, run now, change its time, its runs
System	`administer_system`	Each component's health, the models in use, the version and whether an update exists
Datasets	`administer_system`	Every dataset and its egress — whether its material may be sent to a hosted model engine (section 10)
Demo packs	`administer_system`	Which packs are loaded, and the command that adds one

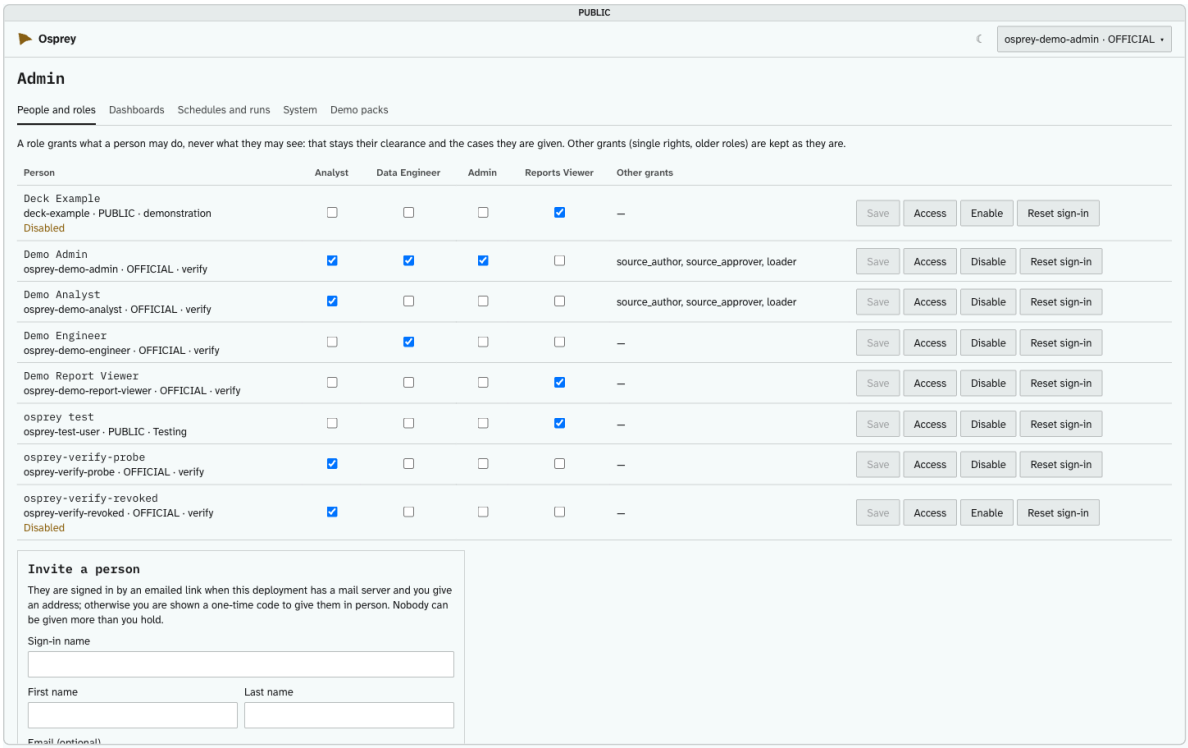


Figure 3. Admin → People and roles: each person's roles, with Access, Disable and Reset sign-in

9.2 Roles

Role	For
Analyst	Cases: investigate, edit the graph, confirm what models proposed, alerts and rules, exports, watches; proposing and approving connection-strength influencers
Data Engineer	Pipelines: build, verify, publish, run and schedule them; the source files they load; held rows; screening lists. (It also carries the right to revert a load, which no screen or command uses yet)
Admin (`osprey-admin`)	The Admin area; the ontology and the map; the audit and export ledgers; proposing and approving connection-strength influencers
Reports Viewer	Reports and data feeds; the alert queue, read-only

A single right can be given on its own. **Nobody can be given more than the person giving it holds** — clearance included — which is why the first administrator holds the highest clearance.

9.3 Inviting someone

Invite asks for a sign-in name, their name, optionally an email, a clearance, a unit, handling codes and roles. With a mail server configured and an email given, they get an email with a link to set their password. Otherwise you are shown a **one-time code** (sixteen letters in four groups) to give them in person; it is shown once, and at first sign-in they choose their own password. **Reset sign-in** works the same way.

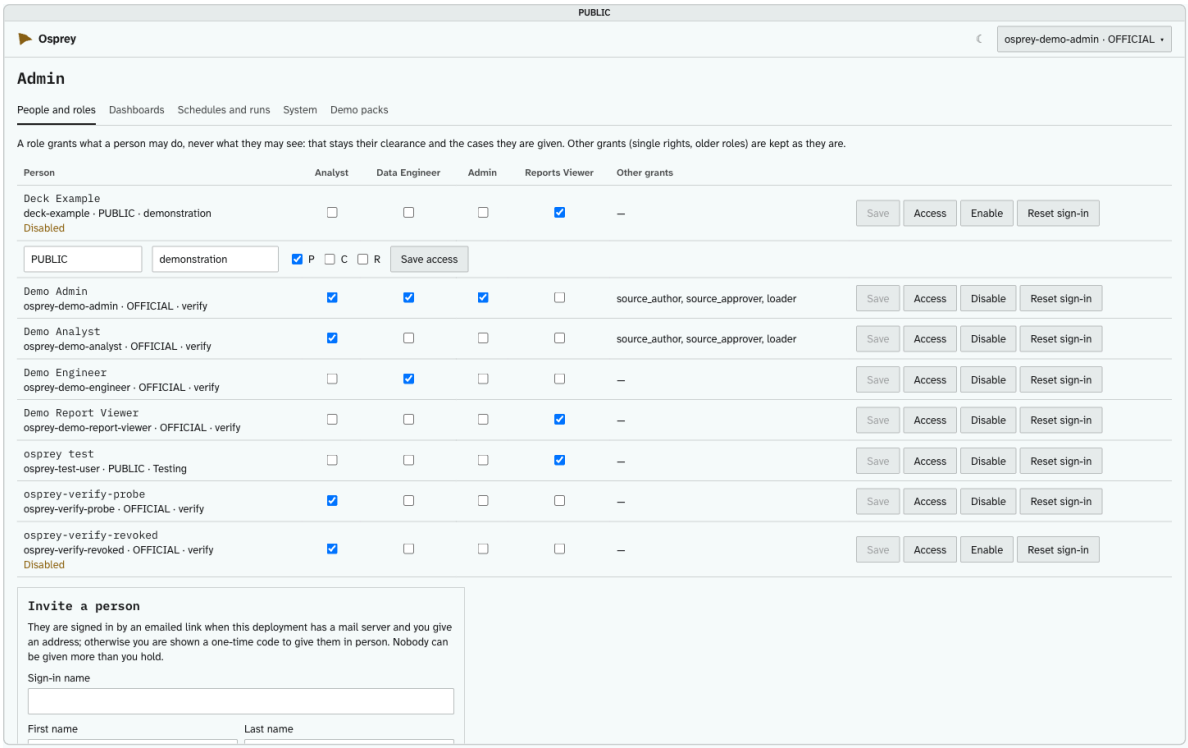


Figure 4. A person's access opened under their row: clearance, unit and handling codes

A mail server is set in `osprey.yaml`'s `mail:` (appendix A); if it signs in, its password is given once at install as `OSPREY_SMTP_PASSWORD` and kept only in `deployment.env`. With mail, the sign-in page also offers *Reset your password*.

9.4 Monitoring: whom a hotlist's alerts reach

Analysts work alerts in one queue, in the Glass's **Monitoring** place: the case's rule alerts reach every member of the case; a watched pattern's alerts reach its owner and the people of the owner's **unit** (the unit set on their account, section 9.3); a hotlist's alerts reach its owner, and the owner's unit too only when the hotlist is shared. Two settings in `osprey.yaml` decide hotlists:

```
monitoring:
  hotlist_audience: only_me        # or team: a new hotlist is shared with its owner's unit
  hotlist_audience_editable: true  # false: owners cannot choose, and a change is refused
```

`hotlist_audience` is where a **new** hotlist starts; it never changes an existing one, and every hotlist made before the setting existed reaches only its owner until the owner changes it. With `hotlist_audience_editable: false`, every new hotlist takes `hotlist_audience`, the Glass offers no choice and says so, and a change is refused in words. A match on data its owner cannot see is never shared. Change either with `ospreyflow apply` (section 8.3). Both are in appendix A.

9.5 Screening lists: subscribing, licences and the refresh

OspreyFlow screens customers against sanctions, PEP and adverse-media lists that the deployment **subscribes** to. Each list is read through its **connector**, which knows one publisher's own export format:

Connector	Reads	Licence (as its publisher states it, 1 November 2026)
OFAC SDN list	`SDN.CSV`, with `ALT.CSV` and `ADD.CSV` beside it when present	A US Government work, in the public domain; OFAC's own pages state no licence
HM Treasury OFSI consolidated list	`ConList.csv`	Open Government Licence v3.0. **Reads a closed list**: the OFSI Consolidated List closed on 28 January 2026 and is no longer updated. Use the UK Sanctions List below for UK designations
UK Sanctions List (FCDO)	`UK-Sanctions-List.xml` (the XML format)	Open Government Licence v3.0 (Crown copyright). The only source of UK sanctions designations since 28 January 2026. Its XML schema carries a notice of its own about commercial use; OspreyFlow does not ship it

Connector	Reads	Licence (as its publisher states it, 1 November 2026)
UN Security Council consolidated list	`consolidated.xml`	**Licence not confirmed.** No open licence found: the UN's terms of use allow personal, non-commercial use. Confirm with the UN before relying on it. Synthetic lists only until then
EU consolidated financial sanctions list	the FSF XML 1.1 file	The European Commission's reuse notice (Decision 2011/833/EU)
OpenSanctions	`entities.ftm.json`	**Licence not confirmed.** CC BY-NC 4.0: **non-commercial only**. Screening inside a business is commercial use and needs OpenSanctions' data licence. Synthetic lists only until then

Check the licence for your use before you subscribe. The architecture's licence register (spec 00 §7) records where each statement was read.

A closed list: OFSI. HM Treasury closed the OFSI Consolidated List on 28 January 2026; the FCDO's UK Sanctions List is now the only source of UK designations. OspreyFlow keeps the OFSI connector, so a file you already hold can still be read, but nothing new will ever be published to it: the Lists tab says "Closed list" on such a list's row, and the subscribe form names the connector "reads a closed list" and points to the UK Sanctions List. To screen against UK designations, download `UK-Sanctions-List.xml` from the list's page (gov.uk/government/publications/the-uk-sanctions-list; the FCDO keeps it at a fixed address, `sanctionslist.fcdo.gov.uk/docs/UK-Sanctions-List.xml`), put it in the ingest folder, and subscribe to it through the **UK Sanctions List (FCDO)** connector. OspreyFlow does not fetch it for you. The connector reads the XML only (its schema is published; the CSV's column order is not).

Licence not confirmed: UN and OpenSanctions. Until their licences for a customer's screening are confirmed, these two connectors read OspreyFlow's own synthetic lists only. The Lists tab marks each such list "Licence not confirmed", with the reason, and the subscribe form says so beside the connector. A real UN or OpenSanctions file is refused, in words, even when this device's exceptions admit its dataset, and nothing loads; `python -m scripts.load_screening_list` refuses a list whose dataset is `opensanctions` or `un_consolidated` the same way.

Real lists and this device. OspreyFlow ships only synthetic lists. A real list loads only when this device's own `config/real_data_exceptions.yaml` (never committed, never copied between machines) names its dataset — for example `ofac_sdn` — for the use the subscription gives; otherwise the refresh is refused, in words, and nothing loads. Add an entry with `python -m scripts.real_data_exceptions add`. A list counts as synthetic only when its file is one of OspreyFlow's own (`packages/demo_fixtures/screening_lists/`, known by its fingerprint); a file cannot make itself synthetic by saying so.

Subscribing. In the Glass, open **Monitoring → Lists** (it needs the `load_screening_lists` right, which the administrator and data-engineer roles hold). Under **Subscribe to a list**, give the list a name, choose its connector, name its file inside the ingest folder (for example `lists/SDN.CSV`), and, for a real list, the dataset it is. A location outside the ingest folder is refused. Put each new download of the list in the same place.

The refresh. Refresh now beside a list (or, on the host, `python -m scripts.refresh_screening_list "<list name>"`, with `--list` to see the subscriptions) reads the file again and applies it as a new **version**: the publisher's own date where the format has one, otherwise the file's fingerprint. OspreyFlow compares it with the version before and re-screens every customer of every case with a screening rule against **the added and changed entries only**, as each rule's deployer and within that case's datasets; each new match is a high-severity alert in that case's queue, its link to the listed party inferred until a person confirms it. A file that is not in the connector's format fails in words and leaves the list as it was. Every refresh is recorded under **Latest refreshes**, with what it added, changed and removed, and how many customers it re-screened and how many alerts it raised **in your own unit**: each screening rule re-screens as its deployer, and its customers and alerts are counted for the deployer's unit alone, so an administrator never learns how many of another unit's customers a list touched. *Refresh now* answers the same way. The audit records who refreshed which list.

When. A list is refreshed when someone presses **Refresh now**, runs the command, or on the list's own **refresh schedule**. Under **Refresh schedule** in Monitoring → Lists, choose the list, **every day** or **every week** (and the day), and the hour (UK time), then **Save the schedule**; **by hand only** stops it. The **Schedule** column shows when each list is refreshed and when next.

- **A scheduled refresh runs as the person who saved the schedule**, resolved afresh each time — never as the system. If they can no longer sign in or no longer hold `load_screening_lists`, that refresh is refused, recorded under **Latest refreshes** in words, and nothing is read; save the schedule again as someone who holds the right.
- Each scheduled time runs once, whichever Conduit replica takes it; after downtime only the latest runs. A refresh that ran on its schedule says so under **Latest refreshes**.
- Put each new download in the list's location before the scheduled hour: the schedule reads the file that is there.

9.6 Approval rights: a second person

Three decisions wait for a second person with the right, and **the person who proposed one can never approve it** (ADR-133 decision 7):

Decision	Who proposes it	Who approves it
Deploying, changing or retiring a rule **for the whole deployment**	An Author of the case who holds `deploy_rules` and `administer_system` (an analyst who is also an administrator)	**Another** person with the same rights and level
Closing a **high-severity** alert another person may see	A member of the case who triages alerts (`triage_alerts`, Editor)	**Another** member who triages alerts in that case
A new version of the **installation-wide connection-strength influencers** (ADR-139)	Anyone holding `manage_strength_influencers` (the Analyst and Admin roles hold it)	**Another** person holding it

A rule for one case, or just for its author, needs no approval. An alert only its owner may see (their own hotlist's, kept to them) is closed by its owner alone, since no second person may read it. Writing a rule as a sentence needs `keep_watches` as well as `deploy_rules`, so a data engineer attaches rules to pipelines (Checks on load) but does not write them.

So a deployment that writes whole-deployment rules needs at least two people holding both the Analyst and the Admin roles (or `deploy_rules` and `administer_system` granted singly), each an Author of a case the rule is listed in (a whole-deployment rule is listed in every case). With one such person, a whole-deployment rule can be proposed but never run.

Approving influencers. In **Monitoring → Connection strength**, a version waiting for a second person shows what it would put in force, who proposed it and why. Another holder of the right chooses **Approve version N**, or **Send it back** with words; its proposer sees the approval withheld and may withdraw it. Until a version is approved, the influencers shipped in the weights file are in force. Every version is kept, with who proposed and who decided it, and each step is in the audit stream. A case's Author may add influencers for that case, or switch off installation-wide ones in it, without a second person; that too is audited. Give the right singly (`right:manage_strength_influencers`) to someone who should set influencers without the rest of a role.

Every proposal, approval and rejection is in the case's rule audit, and every rule keeps its versions — who proposed each and who decided it. **After upgrading to this version**, a whole-deployment rule that was running waits for a second person's approval before it runs again; Monitoring → Rules lists it as waiting, "deployed before a second person's approval was needed".

10. Models

By default every model runs on the deployment itself (extraction, vision, transcription and embedding — appendix A, `models.*`) and **no case content leaves it**. A client with its own agreement with one or more model providers may enable them as further **engines**: `openai`, `anthropic` and `gemini` (Google), and other LiteLLM providers such as `azure` or `bedrock` in the same shape.

```
models:
  local_only: false
  sendable_handling_codes: [P]
  providers:
    - name: gemini
      roles: {extraction: gemini/gemini-2.5-flash}
    - name: openai
      label: OpenAI (client tenancy)
      roles: {extraction: gpt-4o, vision: gpt-4o-mini}
      privacy_contract: true
      contract_reference: DPA 2026/07
  defaults:
    extraction: gemini
```

- **`providers`** lists each enabled provider: its model for each role it takes, an optional `api_base` (`https://` only) and an optional `label` — the name analysts see. Without one, the label is OpenAI, Anthropic or Gemini.
- **`defaults`** names each role's default engine: `local` or a provider's name. A role not named runs on `local`.
- **`privacy_contract`** marks a provider as under a privacy contract: set it to `true` only when your organisation holds a data-processing agreement with the provider that covers this key. It is `false` unless you set it. **`contract_reference`** says which agreement, in words ("DPA 2026/07"); it is shown to analysts, so it is never a secret, and one that looks like a key is refused.
- The older single `provider:` is still read, as a one-entry list whose roles default to it. Naming both `provider` and `providers` is refused.

Keys. Each provider's key is `OSPREY_MODEL_KEY_<NAME>` — for example `OSPREY_MODEL_KEY_GEMINI` (the single `provider:` keeps `OSPREY_MODEL_PROVIDER_KEY`). Give it in the installer's terminal before `ospreyflow install`, or on the guided install's Models screen; an enabled provider without its key stops the install, and a later install keeps a key already given. To enable a provider on a running deployment, add it to `osprey.yaml` and run `ospreyflow apply` (section 8.3), which takes its key from `OSPREY_MODEL_KEY_<NAME>` or asks for it at a hidden prompt. On a running deployment, `ospreyflow models key <name>` sets or replaces a key and `ospreyflow models remove-key <name>` removes it (section 8); `ospreyflow models` says whether each is present. A key is kept in `deployment.env` alone and read only by the deployment's model gateway — never by the Conduit, never written to `osprey.yaml`, never shown in the Admin area or any page of the Glass.

On the guided install page (section 4.1), the Models screen has a key field for each enabled provider. A key typed there is sent only to the install page, on this computer — a request from anywhere else is refused — and is held there until the install gives it to `deployment.env`. It is never written anywhere else, never logged and never shown again: the field empties once the key is sent, and the page says only "key set" or "no key".

What may leave. Material is sent to a provider only when both of these allow it:

- **its handling code** is in `sendable_handling_codes` (one list, for every engine); and
- **every dataset it comes from** allows that provider. Each dataset has an **egress** setting: - `none` — never sent to a hosted engine. **This is every dataset's setting until you change it.** - `contracted` — sent only to a provider marked `privacy_contract: true`. - `any` — sent to any enabled provider.

Anything else runs on the local engine, whatever was chosen, and is refused in words when the deployment has no local model for the role. Material whose datasets are not known is treated as `none`, so nothing leaves by omission. Embeddings never leave the deployment.

Setting a dataset's egress. Only an administrator sets it: Admin → Datasets lists every dataset, the unit that holds it and its egress; choose the setting and press Save beside it. Each change is recorded in the audit log — who made it, when, and from what to what.

> **After upgrading to this release, hosted engines receive nothing** until an administrator sets
>
> a dataset's egress to `contracted` or `any`. Every existing dataset, demo packs included, starts
>
> at `none`; until you change one, every question runs on the deployment's own models, whatever
>
> engine an analyst chooses. Set only the datasets you are entitled to send.

Who chooses. The administrator sets each role's default. An analyst may choose, for each question in Ask the case, among the engines offered — the local engine and each enabled provider that has a key — each shown "under contract" or "no contract". An engine the case's datasets may not reach is shown but cannot be chosen, with the reason beneath the choice. Ask to search offers the same choice: its question goes under the case's handling and from the case's datasets, through the same two checks, and the deployment's own model answers whenever they do not allow the engine chosen. Every answer names the engine and model that produced it.

Admin → System shows each role's model, each engine with its models, the roles it is the default for, whether its key is present and whether it is under a privacy contract (with its reference), and a notice that case content then leaves the deployment. `ospreyflow models` shows the same mark beside each provider ("privacy contract: yes (DPA 2026/07)" or "privacy contract: no"), and the guided install's Models screen shows it as the configuration holds it.

11. Demo packs

Id	What it shows	How it arrives
`hmp`	Prison intelligence: a prison's people and places, cell shares, co-defendants, conflicts, incidents, visits in person and by video, phones, calls and payments (synthetic)	Generated
`fixture-corp`	Network shapes, and duplicate records matched without being merged (synthetic)	Generated
`operation-ledger`	Preventive AML: a structuring ring, onboarding screened before it commits, ownership through a trust (synthetic)	Generated
`prison-intelligence`	Prison intelligence, end to end: a fictional jail's records as an intelligence team receives them, loaded through the Ingest Studio from source files, each with its own marking (synthetic)	Generated
`prison-intelligence-scale`	The same estate at scale: 3,000 prisoners and 30,000 links across six establishments, to show a large network grouped by community (synthetic)	Generated

Id	What it shows	How it arrives
`cyber-intrusion`	Cyber threat analysis in STIX 2.1: a fictional threat actor's campaign against a water company — its backdoor, two command-and-control servers, the indicators that find them and the traffic observed; its case is set to STIX (synthetic)	Generated
`paysim`	Mobile-money fraud: 6.36 million simulated transactions with fraud labels	Downloaded by you
`enron`	**Real data**: about 150 people's real email at Enron, 1998–2002 — who wrote to whom, when, and about what	Downloaded by you; accepted before it loads; never by an install

Which a starter brings is section 1a's table: a starter loads its domain's packs, into cases set to its domain's packs. A **public dataset** of your own choosing, such as a VAST Challenge dataset, is added with `ospreyflow demo add public` (section 1a.2).

Choose them at install (`demo_packs`), or later: `ospreyflow demo list`, `ospreyflow demo add <id>`. Each loads into its own case, with the installing administrator as Author. Adding **HMP** again is safe: a deployment that loaded it before v0.2.0 gains only its prison network, in the same case, and one that already has the network is left as it is. **PaySim** is a download: get `PS_20174392719_1491204439457_log.csv` from its Kaggle page and pass it with `ospreyflow demo add paysim --file <path>`; it is checked by size and checksum before anything starts. PaySim is CC BY-SA 4.0 (E. A. Lopez-Rojas, A. Elmir and S. Axelsson, 2016); its licence and attribution travel with it.

Prison Intelligence seeds its case through the Ingest Studio's own pipelines, as a data engineer would, and has commands for rehearsing a demonstration: `--reset-live` between rehearsals, and two for the session itself (`DEMO_GUIDE.md` in the repository gives each, in full). Turn `components.post_ingest_resolution` off (appendix A) before adding it, or a live load in the demonstration adds automatic "same as" links. Its **scale cohort** (`prison-intelligence-scale`) is written straight into the graph in a case of its own, with no pipeline runs, so the rehearsal reset does not touch it; adding it again changes nothing, and a run that stopped part-way says so rather than finishing it. `python -m scripts.write_pid_files --out <folder> --profile scale` writes its 3,000 prison numbers as `scale_cohort_list.txt`, to paste into *Add many from a list*.

Enron is the one pack of **real personal data**: the email of real people, released during the US regulator's investigation and distributed by Carnegie Mellon University. OspreyFlow never carries it, and an install never adds it. To use it, download `enron_mail_20150507.tar.gz` (423 MB) from https://www.cs.cmu.edu/~enron/ yourself, then:

```
ospreyflow demo add enron --file enron_mail_20150507.tar.gz
```

It checks the file is exactly the May 7, 2015 version (size and SHA-256), then says what it is — real people's names, addresses and private correspondence; **no licence is stated**, CMU distributing it "as a resource for researchers" and asking that you "be sensitive to the privacy of the people involved"; your organisation needs its **own lawful basis** to process it — and loads it only when you type `yes`. (`--accept-real-data` accepts in a scripted run, and says so.) It is unpacked into the demo data folder — 150 mailboxes, about 517,000 messages and 2.5 GB, so allow 3 GB free with the download — with a record of the acceptance beside it, and loads into its own case, "Enron corpus (real data)", shared with no demo account but the administrator and analyst. The terms are in `packages/demo_fixtures/licences/enron/NOTICE.md`.

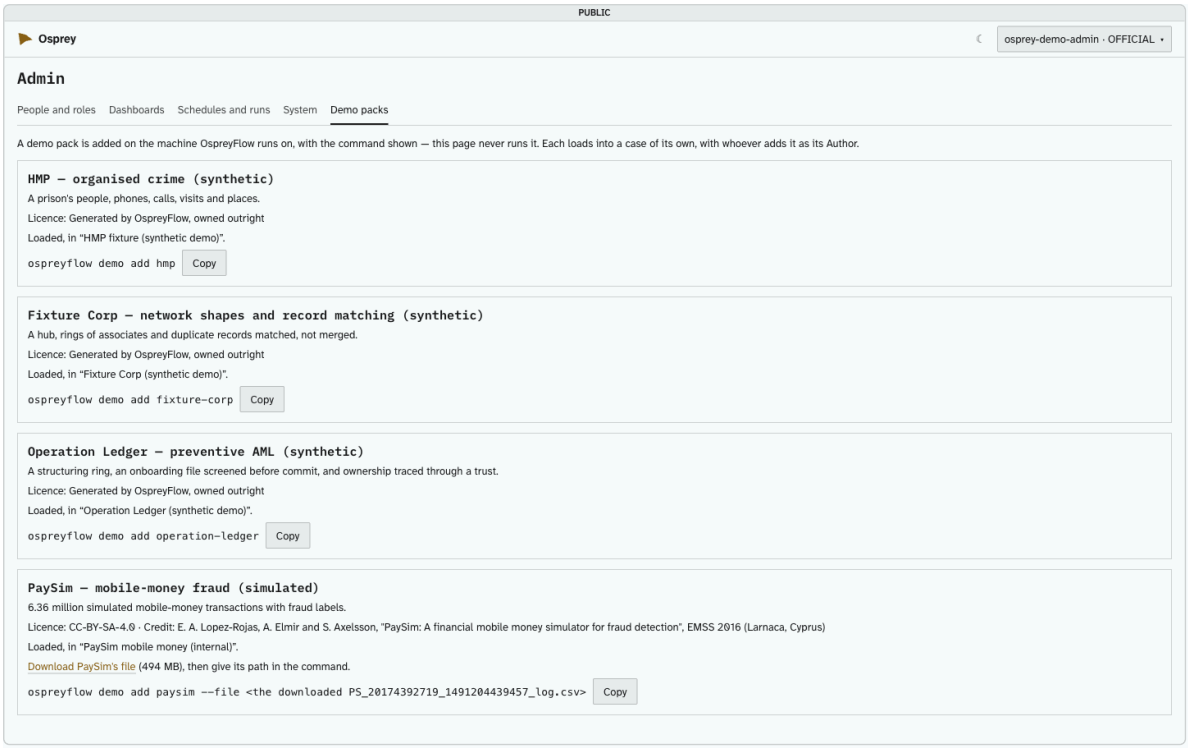


Figure 5. Admin → Demo packs: what each pack shows, where it was loaded, and the command for the rest

12. Dashboards

With dashboards on, **report sync** pulls each case's figures into a reporting database every 15 minutes (`reporting.sync_interval_seconds`), and **Superset** shows them at `bi.`, signing people in through the deployment's identity provider (below). A business intelligence tool (Power BI, Tableau, Qlik) reads the same figures:

1. register the dashboard in Admin → Dashboards and copy its secret (shown once);
2. add it to `REPORT_SYNC_DASHBOARDS` in `deployment.env` (a JSON list of `{"client_id", "secret"}`) and restart: `ospreyflow stop`, `ospreyflow start`;
3. a case's Author gives the dashboard's account Viewer on the case.

Each pull is made as that account, at its clearance, audited and recorded in the export ledger.

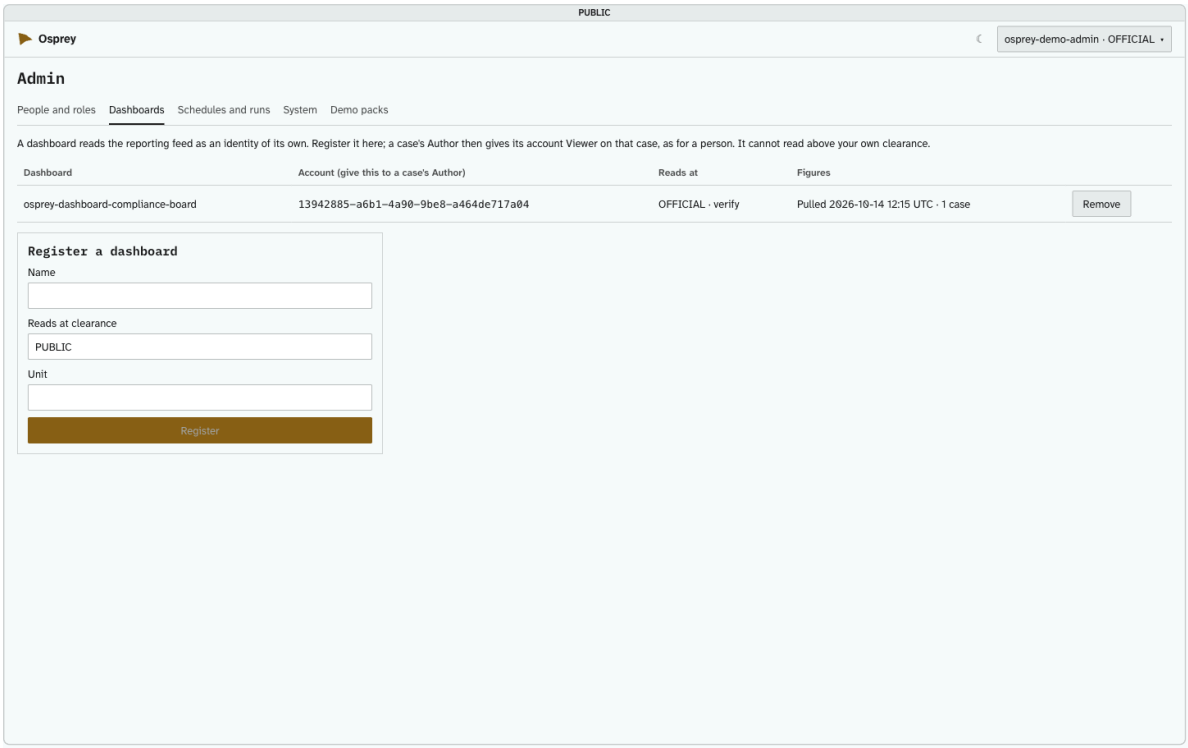


Figure 6. Admin → Dashboards: registering a dashboard's account and seeing when it last pulled

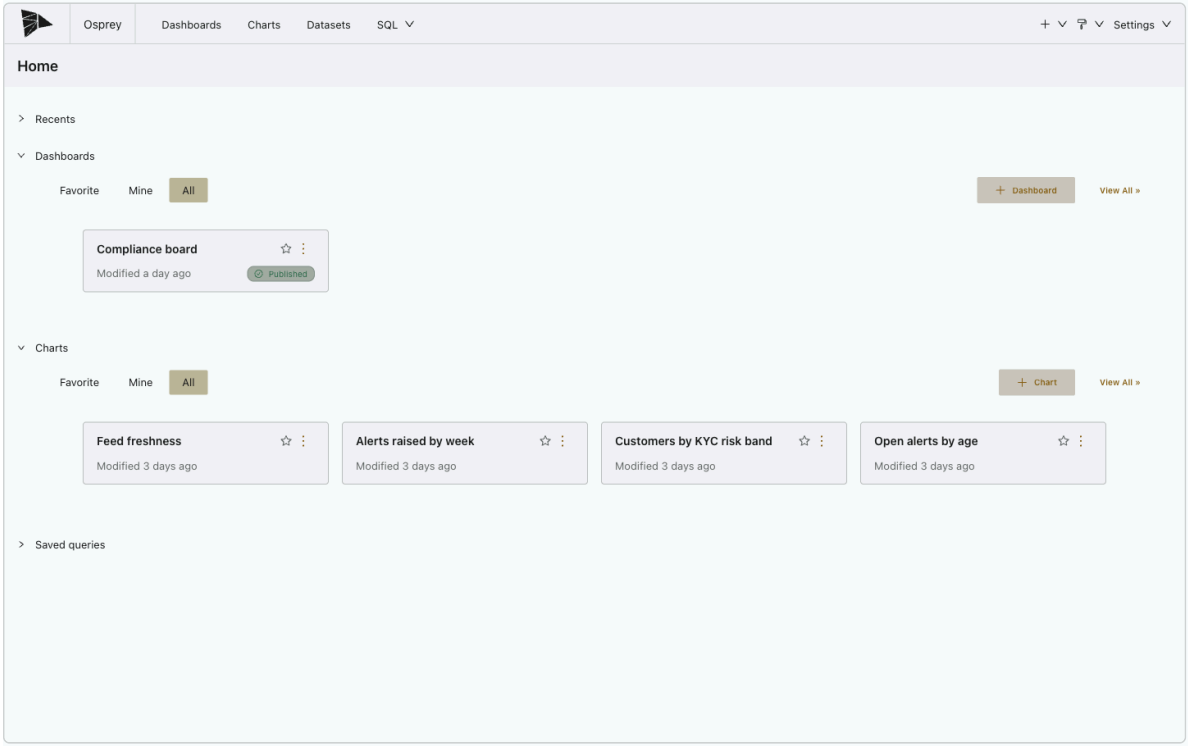


Figure 7. Superset at bi.: the boards and charts built on the reporting figures

The monitoring board

The feed also carries four monitoring sections, for every kind of alert (typology, hotlist and monitor), each a row of `section`, `label`, `typology`, `week_start` and `value` in the reporting database's `figures` table, beside `raised`, `open_by_age` and `outcome`:

`section`	`label`	What it counts
`raised_by_rule`	`typology` (with its `typology`), `hotlists` or `monitors`	alerts raised each week, by rule

section	label	What it counts
closed	the closing reason as stored: true_positive (True match), false_positive (False positive), duplicate, no_further_action or referred; or closed (taken back, no outcome)	alerts closed each week, by outcome
time_to_close	under a day, 1 to 7 days, 7 to 30 days, over 30 days	alerts closed in the period, by how long they took
hotlist_matches	hotlists	hotlist matches each week

A typology rule is named by its typology, never by its own name, which an analyst writes and the feed does not carry. Every figure is counted as the dashboard's account may see it; a hotlist kept "only me" is counted for its owner alone, so never on a dashboard. No board ships as a file: build the monitoring board in Superset from the `figures` table, filtered on these sections — for example a stacked bar of `raised_by_rule` by `week_start` and `label`, a bar of `closed` by `label`, a table of `time_to_close`, and a line of `hotlist_matches` — with a filter on `case_name`. Report sync fills it on its next pull; nothing else needs to change.

13. Backup and restore

Lite. `ospreyflow backup` writes a set to `backups/<time>-<profile>-<commit>/`: the graph store, every PostgreSQL database (the identity provider's accounts as their own export), the analytics database, the object store (each object with its checksum), the configuration, and a manifest of checksums. Taken while the Conduit runs, the stores may be captured at slightly different instants — it says so; `--require-quiesced` refuses to run unless the Conduit is stopped.

`ospreyflow restore <set>` is a **dry run** first: it checks every checksum and estimates how long it will take. `ospreyflow restore <set> --yes` restores. It refuses a set whose checksums do not match, a deployment that is not empty, and a set from another ontology version. Passwords are not restored: the deployment's own are kept.

Small. The cloud protects the data: the database's own backups and snapshots, and the server's daily backup. `ospreyflow backup` refuses a deployment on a managed database, saying so; restore from the cloud's console.

Enterprise. The cloud protects its managed services as for Small. **The graph store and the analytics database run in the cluster, and nothing backs them up yet**: `ospreyflow backup` drives a Compose deployment and does not reach a cluster. Until it does, take snapshots of their volumes with the cloud's own tools.

Audit. The audit log is in the same PostgreSQL as everything else, so it is in the same backup and shares its retention.

14. Updates

Releases are tagged `vMAJOR.MINOR.PATCH`, with signed images and a signed list of every image by digest. Each release's notes are its section of `CHANGELOG.md`.

Which installs `update` moves, and how the others are updated:

Your install	How it is updated
Lite, installed from a release tag	`ospreyflow update` (below)
Lite, built from the checkout (`--build`, or a checkout on no release tag)	`git pull`, then `ospreyflow start --build`. `update` refuses this install, saying so
Small	**Not yet by `update`**: its first step, the backup, refuses a deployment on a managed database (section 13). The cloud's own backups protect the data
Enterprise	Move the checkout to the release's tag and run the install again: the chart is upgraded in place, every password and account kept

Checking for a newer release. The deployment never calls out to look. Run `ospreyflow update --check` on the computer that holds the checkout: it asks the repository's releases and leaves the answer where Admin → System shows it, with when it asked: "*vX.Y.Z* is available", "Up to date", or "The last update check could not answer" and why; before the first check it says "Not checked for updates yet", and an install built from its checkout is told it moves with `git pull` and `ospreyflow start --build`. It checks only when you run it — to be told each week, add it to that computer's own scheduler, for example a cron line `0 8 * * 1 ospreyflow update --check`. If the repository is private, set `GITHUB_TOKEN` to a token that can read it first; without one the check cannot answer, and says so.

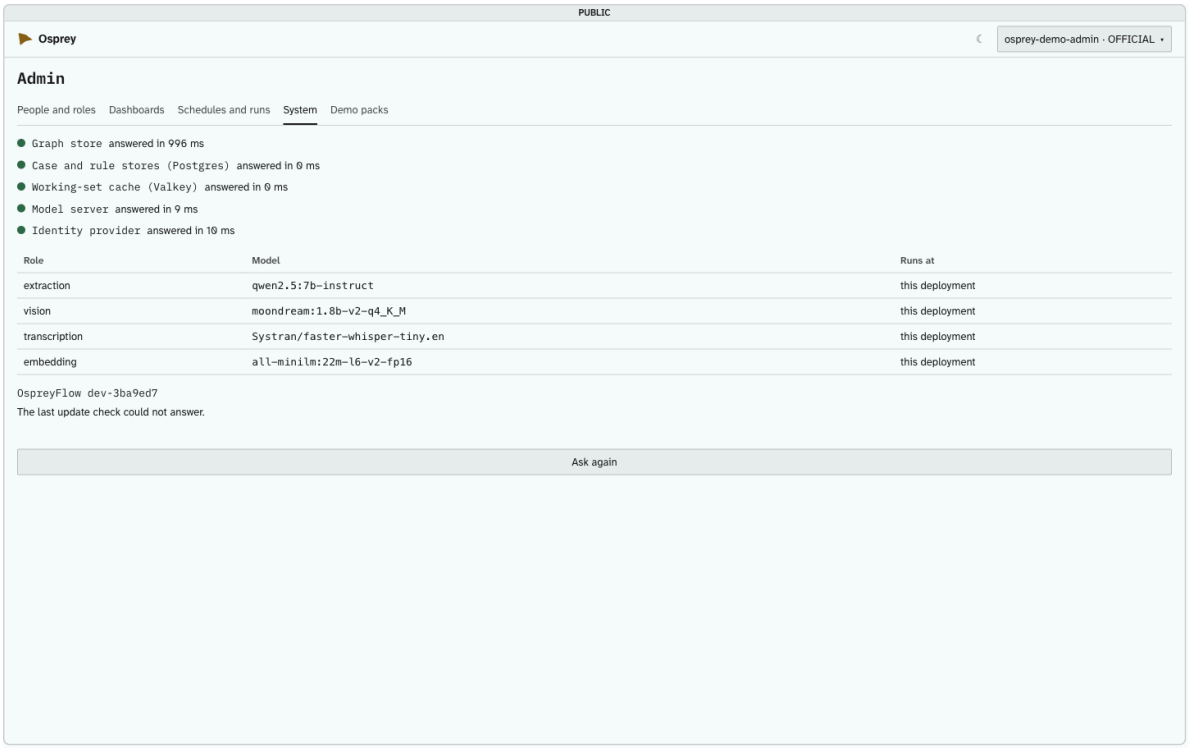


Figure 8. Admin → System: each component's health, the models in use, the version, and the last update check

Moving to a release.

- `ospreyflow update` moves to the newest release (`--to vX.Y.Z` for another; `--yes` asks nothing). It checks every image's signature first and changes nothing if one fails; refuses if a local change to a file would be overwritten ("commit or undo them first"); **backs everything up**; moves the checkout to the tag; starts the release's images and checks each service runs them. If anything fails after the backup it **rolls back** — the previous release and its data restored from the backup just taken — and says so.
- **Without internet**: on a machine with it, `ospreyflow update --make-bundle <file> --to vX.Y.Z` writes the release, verified, to one file; carry it over and run `ospreyflow update --from <file>`, which checks it again with no network.

An update moves forward only.

15. Scheduled and watched pipelines

A published pipeline can run on a **schedule** (hourly, daily, on weekdays or weekly at a time, in a named time zone, or a cron expression; never more often than every 15 minutes) and when a **file arrives** in its dataset's folder (a filename pattern such as `*.csv`; each new file runs once, a minute after it stops changing). Both are set on the Ingest Studio's Publish screen by someone holding `schedule_pipelines`, and run **as that person**, with every check their own run would get. A run that is refused or fails tells its owner in their alerts; three refusals in a row pause it. Admin → Schedules and runs lists every one.

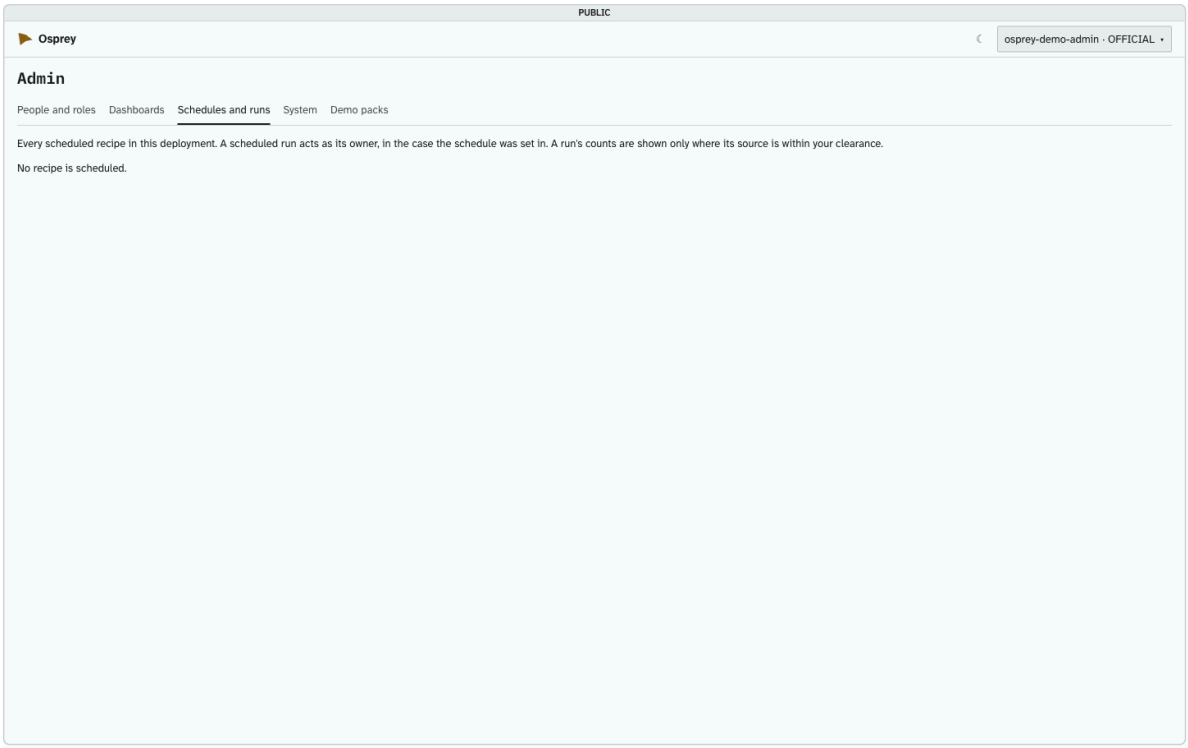


Figure 9. Admin → Schedules and runs

16. When something is wrong

Symptom	Look at
`install` stops at the checks	The crossed line says what to fix; nothing was changed
A service is not healthy	`ospreyflow status`, then `ospreyflow logs <service>`
The page does not open	`ospreyflow status`; that port 80 (Lite) or the DNS records (Small) are right
Nobody can sign in	`ospreyflow logs keycloak`; that the address in `osprey.yaml` is the one in the browser; then `ospreyflow repair-sign-in`
A Small server never answered	Its log, `/var/log/ospreyflow-first-boot.log` (Session Manager on AWS)
An update rolled back	Its own words say why, and where its backup is
The model download stopped with `rename …-partial …: no such file or directory`	A model server restarted mid-download and removed the half-fetched file (fixed from the release after 15 October 2026). Run `ospreyflow start` while the deployment is up: it starts the download again and the server keeps running, so it completes

Appendix A — every `osprey.yaml` setting

Made from the deployment's own schema. Settings marked *section* hold the settings below them.

Option	Value	Default	What it does
`profile`	`local-lite` | `small` | `cluster`	`local-lite`	Which kind of deployment: one machine (local-lite), a cloud server on managed services (small), or a cluster.
`components`	section	—	Which stores the deployment runs, and where their data lives.
`components.graph_store`	`arcadedb` | `janusgraph`	`arcadedb`	The graph store.
`components.analytics_engine`	`duckdb` | `clickhouse`	`duckdb`	Where events are counted: DuckDB inside the Conduit, or ClickHouse shared by several.

Option	Value	Default	What it does
components.post_ingest_resolution	bool	true	Whether a load queues entity resolution afterwards (OSPREY_POST_INGEST_RESOLUTION). False for a demonstration whose people must not gain automatic SAME_AS links; an analyst's own Resolve now still works.
components.search	opensearch | postgres-fulltext	opensearch	Search; postgres-fulltext is the lighter variant.
components.storage	section	—	Folders: inside the containers, and on the host for ingest and tiles.
components.storage.data	str	/var/lib/osprey/data	Where the deployment keeps its data inside the containers.
components.storage.staging	str	/var/lib/osprey/staging	Where loads stage their rows before they reach the graph.
components.storage.backups	str	/var/lib/osprey/backups	Where backups are written.
components.storage.ingest	str	/var/lib/osprey/ingest	The host folder of source files the Ingest Studio lists (read-only to it; uploads go to its uploaded/ folder). A relative path is read from the folder osprey.yaml is in.
components.storage.tiles	str	/var/lib/osprey/tiles	The host folder a built basemap lives in. A relative path is read from the folder osprey.yaml is in.
components.source_path	str or none	none	The owned services (workspace, api, ingest) are PULLED, by the digest recorded in versions.lock, from the registry reference harness/render.py's own service_images names. This is the operator path: the harness image itself carries no services/ or apps/workspace source, and round 05's own finding was that the documented one-command bootstrap could not Launch without it — a clean laptop has no checkout. Set to a checkout root containing services/api/Dockerfile, services/ingest/Dockerfile and apps/workspace/Dockerfile (this repository's own root, for a developer/CI run) to BUILD owned images from source instead. Declared and the Dockerfiles absent is refused by name (harness.render.MissingDockerfileError) — never silently treated as the pull path.
models	section	—	The local models, and the client's own model providers (engines) if any are enabled.
models.local_only	bool	true	No hosted fallback
models.hosted_fallback	str or none	none	A provider name, never a credential.
models.sendable_handling_codes	list of str	—	Which handling codes may leave for a hosted model.
models.provider	section or none	none	The older one-provider form, read as a one-entry providers list; not with providers.

Option	Value	Default	What it does
`models.provider.name`	str	required	LiteLLM's name for the provider: openai, anthropic, gemini, azure, bedrock...
`models.provider.label`	str or none	none	The name analysts see for this engine. Default: OpenAI, Anthropic or Gemini for those three, else the provider's name capitalised.
`models.provider.roles`	mapping	required	The roles it takes, each with the provider's model for it
`models.provider.api_base`	str or none	none	The provider's address, where it is not LiteLLM's default (an Azure deployment, a proxy)
`models.provider.privacy_contract`	bool	false	True when the deployment's organisation holds a data-processing agreement with the provider covering this key. A dataset whose egress is `contracted` is sent only to a provider marked so. Default: false.
`models.provider.contract_reference`	str or none	none	The agreement's reference, in words (for example DPA 2026/07); never a secret. Shown beside the mark.
`models.providers`	list of section	—	The providers (engines) an administrator enables, each reached through the deployment's gateway; each key is OSPREY_MODEL_KEY_<NAME>.
`models.providers.name`	str	required	LiteLLM's name for the provider: openai, anthropic, gemini, azure, bedrock...
`models.providers.label`	str or none	none	The name analysts see for this engine. Default: OpenAI, Anthropic or Gemini for those three, else the provider's name capitalised.
`models.providers.roles`	mapping	required	The roles it takes, each with the provider's model for it
`models.providers.api_base`	str or none	none	The provider's address, where it is not LiteLLM's default (an Azure deployment, a proxy)
`models.providers.privacy_contract`	bool	false	True when the deployment's organisation holds a data-processing agreement with the provider covering this key. A dataset whose egress is `contracted` is sent only to a provider marked so. Default: false.
`models.providers.contract_reference`	str or none	none	The agreement's reference, in words (for example DPA 2026/07); never a secret. Shown beside the mark.
`models.defaults`	mapping	—	Each role's default engine, `local` or an enabled provider's name. A role not named runs on `local` (for the single `provider`, on it).
`models.extraction`	`qwen2.5:7b-instruct` | `qwen2.5:14b-instruct`	`qwen2.5:7b-instruct`	The local model that extracts entities from text. Default: qwen2.5:7b-instruct, the model the product ships with.
`models.vision`	`moondream:1.8b-v2-q4_K_M`	`moondream:1.8b-v2-q4_K_M`	The local model that describes an image and reads its text.

Option	Value	Default	What it does
`models.transcription`	`Systran/faster-whisper-tiny.en`	`Systran/faster-whisper-tiny.en`	The local model that transcribes a video's speech.
`models.embedding`	`all-minilm:22m-l6-v2-fp16`	`all-minilm:22m-l6-v2-fp16`	The local model that turns text into vectors.
`governance`	section	—	Identity, handling, ontology and the marking shown on screen.
`governance.identity_provider`	str	`keycloak`	The identity provider people sign in through.
`governance.handling_scheme`	str	`uk-law-enforcement`	The handling scheme every item is marked under.
`governance.ontology`	str	`default`	The ontology the deployment starts from.
`governance.organisation_sign_in`	str or none	none	The alias of an organisation's own identity provider brokered by Keycloak. When set, the welcome page offers *Continue with organisation sign-in*, which goes straight to it. Default: none — one *Sign in* to the OspreyFlow form.
`governance.marking`	str or none	none	The protective marking shown where no case data is on screen: the welcome page, Home, the Ingest Studio and the Admin area; inside a case, the strip shows this or the highest marking on screen, whichever is higher. One of the handling scheme's classifications. Default: the scheme's lowest.
`governance.real_data_exceptions`	str	`config/real_data_exceptions.yaml`	A path to the gitignored list, never its contents.
`cluster`	section	—	A Kubernetes deployment: its namespace, storage classes and machines.
`cluster.hosts`	list of section	—	One machine.
`cluster.hosts.name`	str	required	The machine's name
`cluster.hosts.address`	str	required	Where the machine is reached
`cluster.hosts.roles`	list of str	—	What runs on it
`cluster.namespace`	str	`ospreyflow`	The Kubernetes namespace the deployment is installed in
`cluster.storage_class`	str or none	none	The storage class each store's own volumes use; the cluster's default when unset
`cluster.shared_storage_class`	str or none	none	Required for a cluster: the storage class of the one volume every Conduit shares (ReadWriteMany) — EFS on AWS, Azure Files on Azure
`network`	section	—	The address the deployment answers at, and what it publishes.
`network.expose_conduit`	bool	false	Publish the Conduit's own port on the host in addition to the workspace's. Not needed to use the deployment — the workspace proxies /v1 — and turning it on widens the exposed surface by a whole tier. For an operator debugging the Conduit directly, or a second client that is not the Glass.

Option	Value	Default	What it does
`network.public_origin`	str	`http://ospreyflow.localhost`	The origin a browser uses to reach the Glass; the identity provider is `id.` and the dashboards `bi.` in front of the same host, with the same scheme and port, and the front proxy publishes that port. `06_DECISIONS.md` ADR-050 objection 2's resolution: the identity-provider realm is rendered from this value at apply time and the Glass's public client is registered with an exact redirect URI and post-logout URI derived from it — no wildcards. Change this to move the deployment to another name or port; nothing else needs to agree with it.
`map`	section or none	none	The basemaps the map offers.
`map.default`	str	required	The provider the map opens on, by its id
`map.providers`	list of section	required	One basemap.
`map.providers.id`	str	required	The provider's own name, as map.default names it
`map.providers.label`	str	required	What the Glass's map menu calls it
`map.providers.kind`	`vector` | `raster` | `image` | `dem`	required	
`map.providers.licence`	section	required	The basemap's licence.
`map.providers.licence.id`	str	required	The licence's identifier, e.g. ODbL-1.0 or OGL-UK-3.0
`map.providers.licence.redistributable`	bool	required	Whether the tiles may be passed on to others
`map.providers.licence.client_scope`	`open` | `per-deployment` | `per-seat`	required	Who the licence covers: anyone, this deployment, or each seat
`map.providers.licence.note`	str or none	none	Anything the licence asks to be said
`map.providers.source`	str or none	none	Where the tiles are: a file (file://… pmtiles) or a provider's address; one of source, service and endpoint, or profile
`map.providers.service`	`wmts` | `wms` | `wfs` | `ogc-features` or none	none	
`map.providers.endpoint`	str or none	none	An OGC service's base address (with service), never a tile URL
`map.providers.layer`	str or none	none	The service's layer to show
`map.providers.tile_matrix_set`	str or none	none	WMTS: the tile matrix set
`map.providers.format`	str or none	none	WMTS: the tile image format
`map.providers.wmts_style`	str or none	none	WMTS: the service's STYLE
`map.providers.zoom_min`	int or none	none	The lowest zoom the provider serves
`map.providers.zoom_max`	int or none	none	The highest zoom the provider serves
`map.providers.profile`	str or none	none	A ready-made provider (providers/<profile>.yaml) to use
`map.providers.variables`	mapping	—	Values the profile's addresses are filled with
`map.providers.credential`	str or none	none	The provider's key, as env:VARIABLE — a name, never the key itself

Option	Value	Default	What it does
`map.providers.proxy`	`required` | `recommended` | `none`	`none`	Whether tiles are fetched through the Conduit (which caches them and keeps the key on the server) rather than by the browser
`map.providers.cache`	section or none	none	Keeping fetched tiles a while (online providers only).
`map.providers.cache.ttl_days`	int	required	How long a fetched tile is kept, in days
`map.providers.cache.prefetch_areas`	list of str	—	Areas whose tiles are fetched ahead of use
`map.providers.style`	str or none	none	The map style: bundled:osprey-dark, bundled:osprey-light, or the profile's own
`map.providers.attribution`	str	required	The credit shown on the map, as the licence asks
`reporting`	section	—	Dashboards for the business: report sync and Superset.
`reporting.sync`	bool	false	Run report-sync
`reporting.sync_interval_seconds`	int	`900`	How often each case is pulled into the mart, in seconds.
`reporting.sync_memory`	str	`256m`	Report sync's memory limit.
`reporting.superset`	bool	false	Run Superset
`reporting.superset_memory`	str	`2g`	Superset's memory limit.
`monitoring`	section	—	Monitoring and alerts: whom a new hotlist's alerts reach, and who changes it.
`monitoring.hotlist_audience`	`only_me` | `team`	`only_me`	Whom a new hotlist's alerts reach: its owner alone (only_me), or also the owner's team, the people of the owner's organisational unit (team). Applies to hotlists made after it is set, never to an existing one.
`monitoring.hotlist_audience_editable`	bool	true	Whether a hotlist's owner may choose its audience, when making it or later. When false, every new hotlist takes hotlist_audience and a change is refused.
`mail`	section or none	none	A mail server, for invitations and password resets by email.
`mail.host`	str	required	The mail server's name
`mail.port`	int	`587`	The mail server's port.
`mail.from_address`	str	required	The address invitations come from
`mail.starttls`	bool	true	Whether mail is sent over TLS (STARTTLS).
`mail.username`	str or none	none	A relay that takes mail without signing in. With one, the password comes from OSPREY_SMTP_PASSWORD at install.
`welcome`	section	—	The welcome page before sign-in: its About section and footer links.
`welcome.about`	bool	true	Off for an internal-only deployment.
`welcome.account_help`	str or none	none	Where people get help with their account.
`welcome.accessibility`	str or none	none	The organisation's accessibility statement.
`welcome.privacy`	str or none	none	The organisation's privacy notice.

Option	Value	Default	What it does
`welcome.service_status`	str or none	none	A page saying whether the service is up.
`deployment`	section	—	Which deployment this is, on a machine that runs more than one.
`deployment.name`	str or none	none	Names this deployment's Compose project, its volumes and its rendered folder, so a second deployment on the same machine shares none of them.
`deployment.graph_port`	int or none	none	The port on this machine (127.0.0.1 only) the graph store's own tools reach it at. A deployment installed beside another with ospreyflow install --name is given one of its own, so the two share no port.
`appearance`	section	—	How every page looks: the deployment's brand, and what a GOV.UK header names.
`appearance.brand`	`ospreyflow` | `govuk`	`ospreyflow`	The GOV.UK look, for a service on a gov.uk domain only.
`appearance.service_name`	str or none	none	The service's name in the GOV.UK header and the page title.
`appearance.organisation`	str or none	none	The organisation in the GOV.UK footer and the page title.
`appearance.phase`	`alpha` | `beta` | `live` | `none`	`none`	The phase banner shows at alpha or beta.
`external`	section	—	What a cloud runs for the deployment instead of a container (Small).
`external.postgres`	section or none	none	A cloud's managed PostgreSQL; its password arrives at install.
`external.postgres.host`	str	required	The managed server's address
`external.postgres.port`	int	`5432`	Its port
`external.postgres.database`	str	`ospreyflow`	The deployment's database on it
`external.postgres.user`	str	`osprey`	The login, which needs CREATEROLE and CREATEDB there
`external.postgres.sslmode`	`require` | `verify-ca` | `verify-full`	`require`	A managed server is reached over TLS.
`external.object_store`	section or none	none	A cloud's S3, for every S3-protocol store (AWS).
`external.object_store.endpoint`	str	required	Host and port, no scheme: always HTTPS
`external.object_store.region`	str	required	The region the buckets live in
`external.object_store.credentials`	`instance` | `keys`	`instance`	The server's own cloud identity, no key anywhere; `keys` reads OSPREY_S3_ACCESS_KEY and OSPREY_S3_SECRET_KEY at apply.
`external.object_store.bucket_prefix`	str	`ospreyflow-`	Each bucket is <prefix><name>, as OpenTofu made them
`external.media`	section or none	none	Media on Azure Blob; its key arrives at install.
`external.media.blob_account`	str	required	The Blob storage account
`external.media.blob_container`	str	`media`	The container media is kept in
`external.valkey`	section or none	none	A cloud's managed Valkey or Redis (cluster only).

Option	Value	Default	What it does
`external.valkey.host`	str	required	The managed cache's address
`external.valkey.port`	int	`6379`	Its port
`external.valkey.tls`	bool	true	Whether it is reached over TLS
`external.search`	section or none	none	A cloud's managed OpenSearch (cluster on AWS only).
`external.search.url`	str	required	The domain's HTTPS address